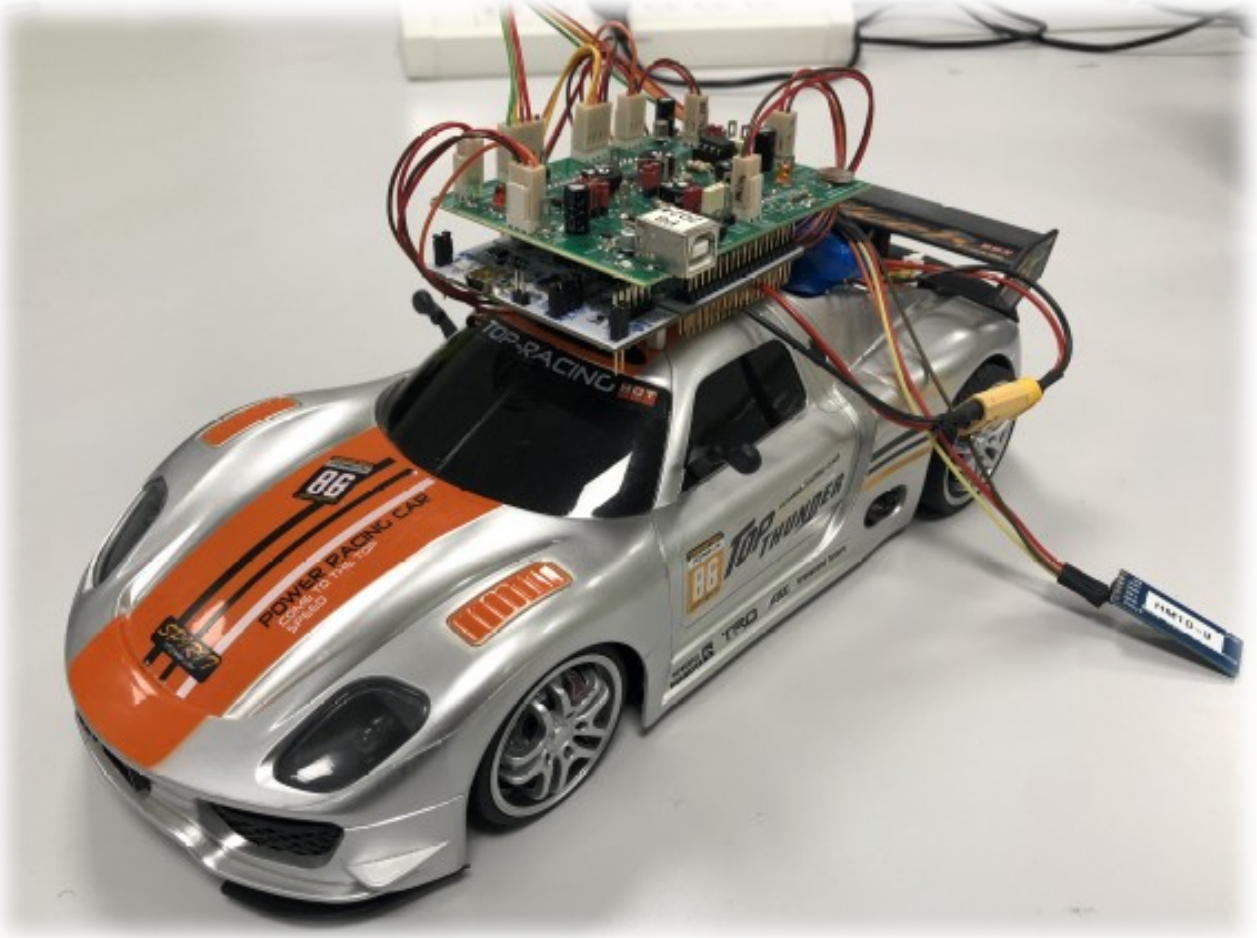
Cahier de Laboratoire
Partie
Informatique Embarquée



NGANJI MUCYO Paguiel

Groupe 2

Année 2025/2026

IUTA Dép. GEII

Sommaire

Table des matières

A. Le cahier d'exigences 3

B. Analyse fonctionnelle du programme 4

1) Structure du programme 4

2) Les fonctions 5

a) Gestion bargraphe 5

b) Gestion buzzer 5

c) PWM direction 5

d) PWM propulsion 6

e) Gestion des phares 6

f) Gestion température 6

g) Gestion bargraphe améliorée 7

h) Gestion capteur avant 7

i) Gestion sens avant 7

j) Gestion buzzer améliorée 7

k) Gestion propulsion améliorée 7

l) Gestion buzzer par sortie PWM 7

C. Conception de chaque fonction 8

1) Fonction `gestion_bargraphe` 8

2) Fonction `gestion_buzzer` 9

3) Fonction `pwm_direction` 10

D. Mise en œuvre 11

1) Fonction `gestion_bargraphe` 11

2) Fonction `gestion_buzzer` 11

3) Fonction `pwm_direction` 12

E. Tests unitaires 13

1) test unitaire de la fonction `gestion_bargraphe` 13

2) test unitaire de la fonction `gestion_buzzer` 14

3) test unitaire de la fonction `pwm_direction` 17

F. Test système complet 19

Annexes 20

Schémas Electriques 21

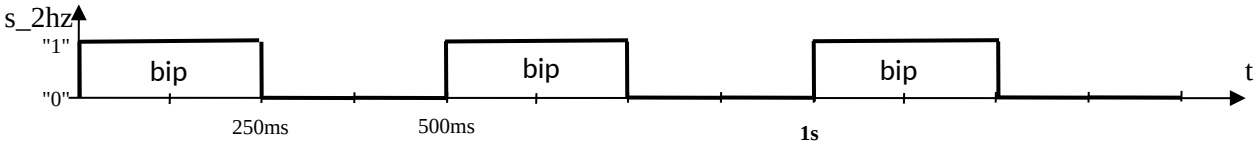
Schéma d'Implantation de la carte Shield 22

A. Le cahier d'exigences

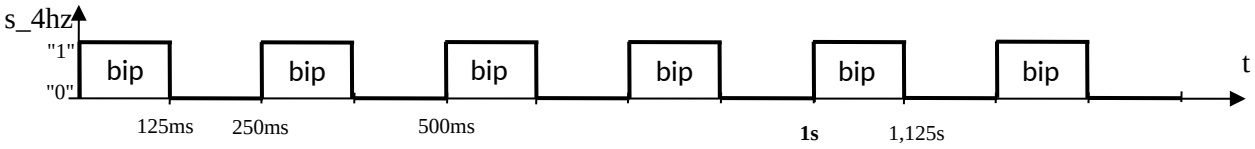
Le cahier des charges du client a donné lieu à des équivalents ou traductions dans le domaine technique. Ces équivalents ou traductions s'appellent des exigences techniques. Ces exigences techniques sont regroupées dans le cahier d'exigences (première étape du cycle en V).

Le cycle en V est une méthodologie de conception pour mener à bien un projet à partir du cahier des charges du client.

- L'utilisateur doit avoir une indication visuelle du niveau des batteries
- Le buzzer doit bipper :
  - bip continu ou mélodie lorsque l'utilisateur appuie sur le bouton klaxon du smartphone. Le klaxon est prioritaire sur toutes les émissions sonores.
  - 2 bips par seconde lorsque la voiture est dans la zone "moyenne distance" à l'avant ou à l'arrière



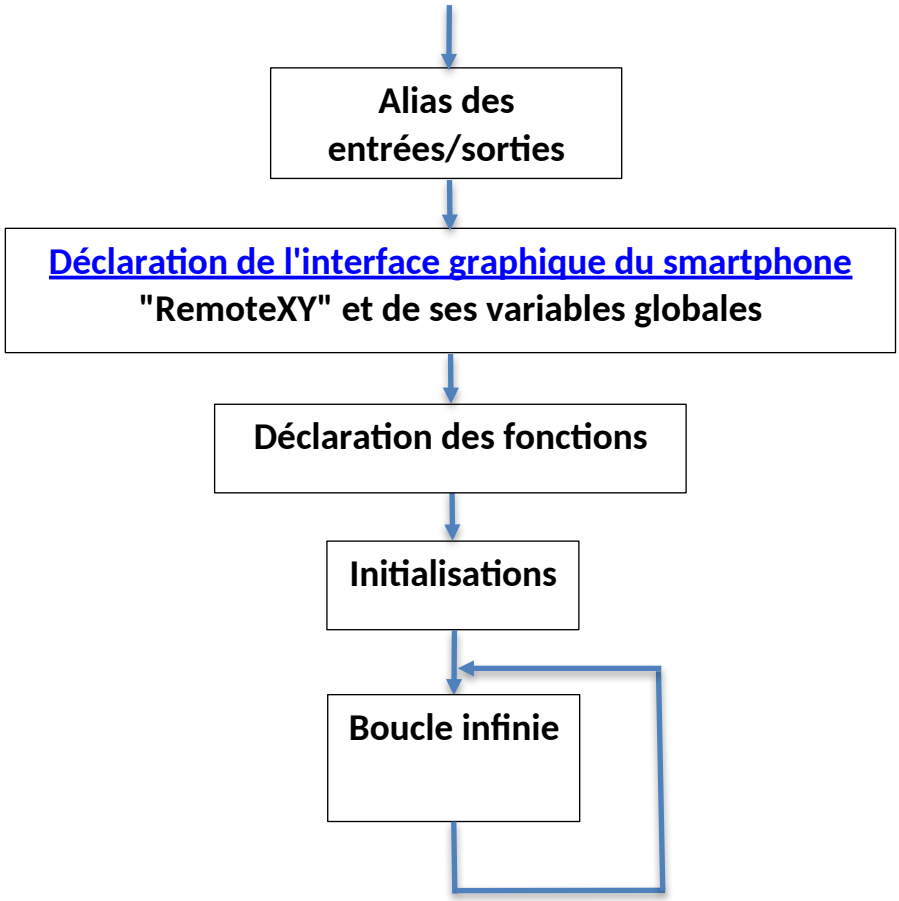
  - 4 bips par seconde lorsque la voiture est dans la zone "près" à l'avant ou à l'arrière.



- Le joystick sur l'axe X doit piloter la direction du véhicule de façon proportionnelle. Une zone de ±10% au centre du joystick sera sans action sur la direction.
- Le joystick sur l'axe Y doit piloter la propulsion du véhicule de façon proportionnelle si la liaison Bluetooth est active. Une zone de ±10% au centre du joystick sera sans action sur la propulsion.
  La consigne de vitesse doit être réduite de moitié dans la zone "moyenne distance".
  La consigne de vitesse est réduite à 0 dans la zone "près".
  Le freinage est maintenu tant que le véhicule se rapproche de l'obstacle dans la zone "près" uniquement (consigne de vitesse maximale en sens inverse durant le freinage).
- Les phares arrière s'allument à 30% avec les phares avant et à 100% au moment d'un freinage
- Les phares s'allument automatiquement en cas de faible luminosité en position AUTO
- Le signal d'approche d'obstacle en marche avant est calculé toutes les 100 ms à partir de la position moyenne avant.
- La position moyenne avant est calculée toutes les 10 ms à partir du signal du capteur sur 10 échantillons (un échantillon toutes les 10 ms).
- Un bargraphe sur le smartphone indique le niveau de la batterie ou la proximité avec l'obstacle avant si celui-ci est dans la zone "près".
- Le capteur de température est interrogé toutes les 3 secondes. La valeur de la température est alors affichée sur l'écran du smartphone.

B. Analyse fonctionnelle du programme

1) Structure du programme



Alias des entrées/sorties :
Cette partie du programme permet de faire le lien entre le nom des signaux électriques (voir schéma électrique) et la numérotation des entrées/sorties du microcontrôleur.
Exemple environnement Arduino : l'entrée/sortie A1 est associée au signal électrique "distance avant".

Déclaration de l'interface graphique :
Cette partie correspond à la déclaration de l'interface graphique du smartphone et de ses variables globales. Voir le fichier d'utilisation de remoteXY.

Déclaration des fonctions :
Cette partie est réservée à la déclaration de l'ensemble des fonctions.
Les lignes de code sont commentées pour une meilleure collaboration entre intervenants.

Initialisation :
Cette partie est réservée à l'exécution d'instructions qui n'ont besoin d'être exécutées qu'une seule fois et en début de programme.
Ex : Instruction pour définir une broche du microcontrôleur en entrée ou en sortie

Boucle infinie :
Cette partie du programme est exécutée en boucle.
Les informations des entrées sont recueillies, les différentes fonctions sont appelées ...

2) Les fonctions

a) Gestion bargraphe

Rôle de la fonction :

Cette fonction permet, à partir de la conversion de la tension batterie, d'indiquer sur un bargraphe à l'écran du smartphone le niveau de batterie.
Au-dessus de 3 V la batterie est pleine.
En dessous de 2.5 V la batterie est vide.
Entre 2.5 V et 3 V le pourcentage de la batterie évolue linéairement en fonction de la tension

Vue externe de la fonction :



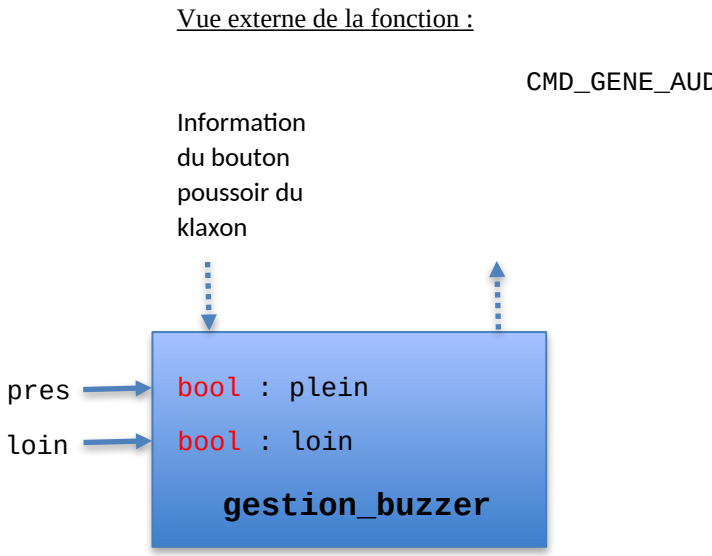
b) Gestion buzzer

Rôle de la fonction :

Dans la version de base sans mélodie, cette fonction permet d'élaborer un signal de commande de l'oscillateur (CMD_GENE_AUDIO) afin d'autoriser un signal sonore (bip) dans plusieurs cas de figure :

1) bip continu si appui sur le bouton Klaxon du smartphone . Le klaxon est prioritaire.
2) 2 bips par seconde lorsque la voiture est dans la zone moyen à l'avant ou à l'arrière.
3) 4 bips par seconde lorsque la voiture est dans la zone près à l'avant ou à l'arrière.

Vue externe de la fonction :



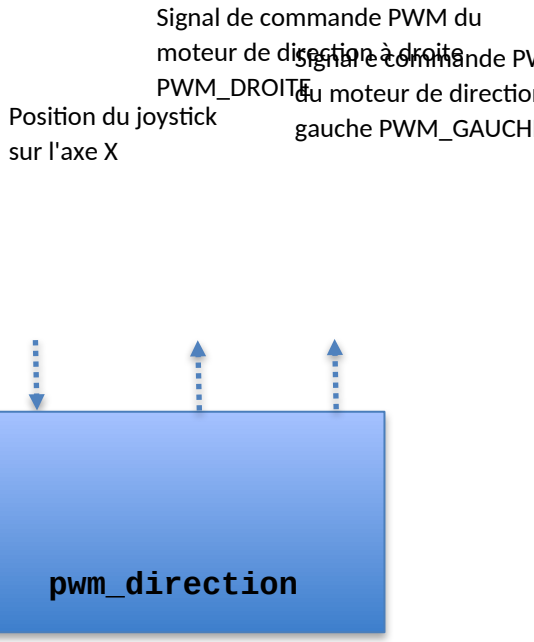
c) PWM direction

Rôle de la fonction :

Cette fonction permet, à partir des informations de position du joystick sur l'axe X, de délivrer 2 signaux PWM.

Un signal PWM pour tourner à droite et un autre pour tourner à gauche.

Vue externe de la fonction :



Remarques :
Cette fonction est pilotée par la valeur que renvoie le joystick sur l'axe X (-100 ≤ valeur de l'axe X ≤ 100)
La sortie PWM_DROITE commande la direction à droite
La sortie PWM_GAUCHE commande la direction à gauche
PWM_DROITE et PWM_GAUCHE ont une valeur comprise entre 0 et 100 (0 la roue est dans l'axe de la voiture, 100 la roue est tournée au maximum.

Afin de minimiser la consommation des moteurs, seules les consignes supérieures à 10 ou inférieures à -10 seront prises en compte.

d) PWM propulsion

Rôle de la fonction :

Cette fonction permet, à partir des informations de position du joystick sur l'axe Y, des informations `pres` et `loin` du capteur arrière et de la vitesse d'approche de l'obstacle à l'arrière, de délivrer 2 signaux PWM et de retourner une variable `freinage` qui servira dans une autre fonction pour allumer les feux arrière.
La consigne de vitesse est réduite de moitié dans la zone "moyenne distance" pour minimiser la distance de freinage dans la zone "près".
La consigne de vitesse est forcée à 0 dans la zone "près".
Le freinage est maintenu tant que le véhicule se rapproche de l'obstacle dans la zone "près" uniquement (consigne de vitesse maximale en sens inverse durant le freinage jusqu'à arrêt du véhicule).

Deux signaux PWM sont à générer : un pour la marche avant et un autre pour la marche arrière.

En cas de perte du signal bluetooth la consigne de vitesse est forcée à 0 afin d'empêcher la destruction du véhicule.

Remarques :
Cette fonction prend en compte les informations de zone `pres` et `loin` du capteur arrière ainsi que l'information `sens_ar` qui prévient d'un rapprochement avec un obstacle en marche arrière.
2 autres champs sont utilisés à partir de la structure globale `RemoteXY` comme :

- La valeur que renvoie le joystick sur l'axe y
  (-100 ≤ valeur de l'axe Y ≤ 100)
- L'indicateur de non-perte de liaison Bluetooth

La sortie PWM_AVANT commande la propulsion avant si PWM_ARRIERE = 0
La sortie PWM_ARRIERE commande la propulsion arrière si PWM_AVANT = 0

Les rapports cycliques des sorties PWM_AVANT et PWM_ARRIERE ont une valeur comprise entre 0 et 100 (0 les roues sont à l'arrêt et 100 les roues tournent à vitesse maximale)
Afin de minimiser la consommation des moteurs, seules les consignes supérieures à 10 ou inférieures à -10 seront prises en compte.

e) Gestion des phares

Rôle de la fonction :

Dans cette fonction, les phares arrière s'allument à 30% (PWM) lorsque les phares avant sont allumés et à 100% si freinage même si les phares avant sont éteints. L'indication de freinage est fournie par la fonction propulsion.

Les phares s'allument automatiquement en cas de faible luminosité. Le seuil entre nuit et jour est de 1.5 V (revoir le chapitre capteur de luminosité de la partie électronique).

f) Gestion température

Rôle de la fonction :

- Cette fonction permet un affichage de la température extérieure sur le SMARTPHONE.
- L'adresse du capteur est personnalisée (voir les 3 cavaliers sur la carte pour l'adresse personnalisée)
- Une amélioration de la fonction est que la température ne soit rafraichie que toutes les 3 secondes.

g) Gestion capteur avant

Rôle de la fonction :

Cette fonction permet de faire l'acquisition de la distance de l'obstacle à l'avant.
Pour ne pas être perturbé par les parasites électriques et obtenir une valeur suffisamment stable cette fonction fait l'acquisition de la valeur du capteur toutes les 10 ms et la range dans un tableau de 10 valeurs avec un indice (index) glissant, puis fait la moyenne sur les 10 valeurs du tableau et retourne cette valeur qui sera utilisée par ailleurs.

h) Gestion bargraphe améliorée

Rôle de la fonction :

Cette fonction reprend le fonctionnement du bargraphe du début. La proximité d'un obstacle avec le capteur avant est indiqué sur le bargraphe à la place du niveau de batterie lorsque la voiture est dans la zone "près".
Dès que la voiture sort de la zone "près", à l'avant, le bargraphe reprend son fonctionnement normal de l'affichage du niveau de batterie.

i) Gestion sens avant

Rôle de la fonction :

Cette fonction utilise la valeur moyenne retournée par la fonction `gest_capt_avant` pour retourner, elle-même, une valeur indiquant une approche d'obstacle à l'avant.
Toutes les 100 ms une comparaison est effectuée. Si la valeur moyenne de la position a augmenté de plus de 5 alors le signal d'approche retourné par la fonction vaut `true` sinon `false`.

j) Gestion buzzer améliorée

Rôle de la fonction :

Cette fonction reprend la fonction buzzer du début et prend en compte la proximité d'obstacle à l'avant.
Optionnel : Les bips avant et arrière ne sont émis que si le véhicule est en approche de l'obstacle dans la zone considérée.

k) Gestion propulsion améliorée

Rôle de la fonction :

Cette fonction reprend la fonction `pwm_propulsion` et prend en compte la proximité d'obstacle à l'avant dans l'anticollision.

l) Gestion buzzer par sortie PWM

Rôle de la fonction :

Cette fonction reprend toutes les fonctionnalités de la fonction `gestion_buzzer()`, sans utiliser l'oscillateur de la carte Shield mais en utilisant les fonctionnalités du PWM de la sortie TONE repérée en D10 (ne pas oublier de déplacer le cavalier en CAV2 pour permettre au signal TONE de piloter le buzzer).

Dans un premier temps générer un bip à 3,5 kHz et un klaxon à 2 kHz.

Pour les plus avancés voici la séquence du klaxon italien "La CUCARACHA" :
Suite des fréquences, en Hz, à émettre en non bloquant pendant 120 ms (chaque fréquence) entrecoupée de blancs de 30 ms :
523, 523, 523, 698, 880, 523, 523, 523, 698, 880,698, 698, 659, 659, 587, 587, 523

C. Conception de chaque fonction

Vue externe de la fonction :

1) Fonction `gestion_bargraphe`

Niveau bargraphe



Rôle de la fonction :

Cette fonction permet, à partir de la tension batterie, d'indiquer sur un bargraphe à l'écran du smartphone le niveau de batterie.
Au-dessus de 3 V la batterie est pleine.
En dessous de 2.5 V la batterie est vide.
Entre 2.5 V et 3 V le pourcentage de la batterie évolue linéairement en fonction de la tension

Calcul et dimensionnement :

Par défaut, le CAN possède une résolution de 10 bits

Nous le programmons sur 12 bits. Le CAN convertit numériquement une tension de 0V à 3.3V

0V => 11111111111 en binaire => 4065 en décimal

Conversion de 2.5V et 3V en décimal :

2.5V => 10.1

3V => 11

Pour trouver l'équation d'une droite passant par deux points :

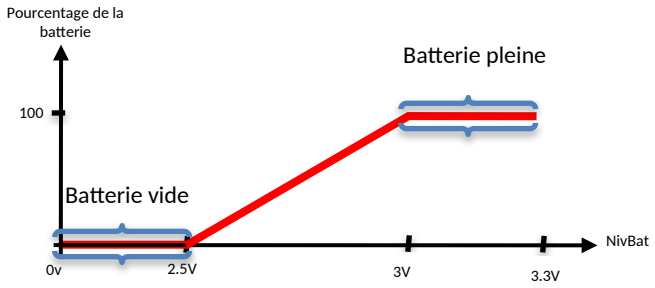
Equation de droite :

1. Calcul du coefficient directeur ()

2. Calcul de l'ordonnée à l'origine ()

3. L'équation finale

Algorithme ou algorigramme de la fonction :



CMD_GENE_A

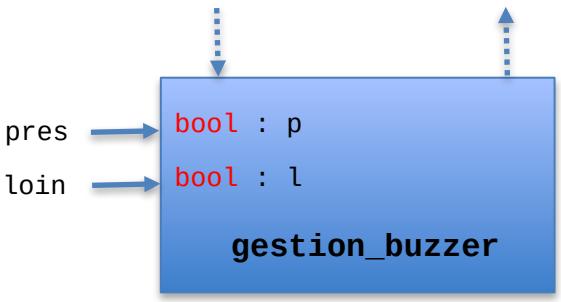
Vue externe de la fonction :

2) Fonction `gestion_buzzer`

Rôle de la fonction :

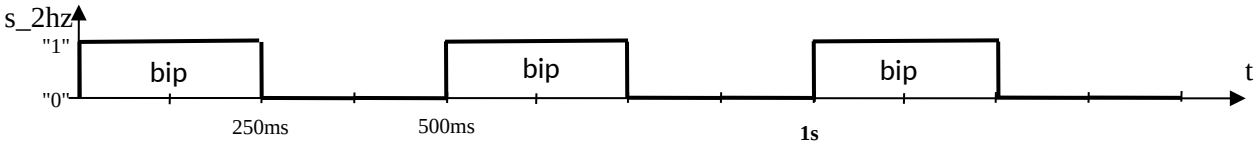
Cette fonction permet d'élaborer un signal de commande de l'oscillateur (CMD_GENE_AUDIO) afin d'émettre un signal sonore (bip) dans plusieurs cas de figure :

- bip continu si appui sur le bouton Klaxon du smartphone. **Le klaxon est prioritaire.**
- 2 bips par seconde lorsque la voiture est dans la zone moyen à l'avant ou à l'arrière.
- 4 bips par seconde lorsque la voiture est dans la zone près à l'avant ou à l'arrière

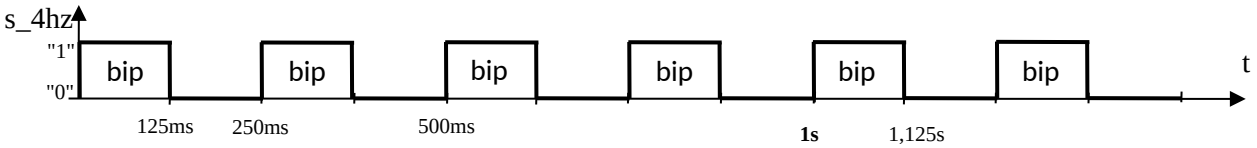


Dimensionnement :

- Le buzzer doit bipper :
  - bip continu lorsque l'utilisateur appuie sur le bouton klaxon du smartphone. Le klaxon est prioritaire sur toutes les émissions sonores.
  - 2 bips par seconde lorsque la voiture est dans la zone "moyenne distance"



  - 4 bips par seconde lorsque la voiture est dans la zone "près" à l'avant ou à l'arrière.



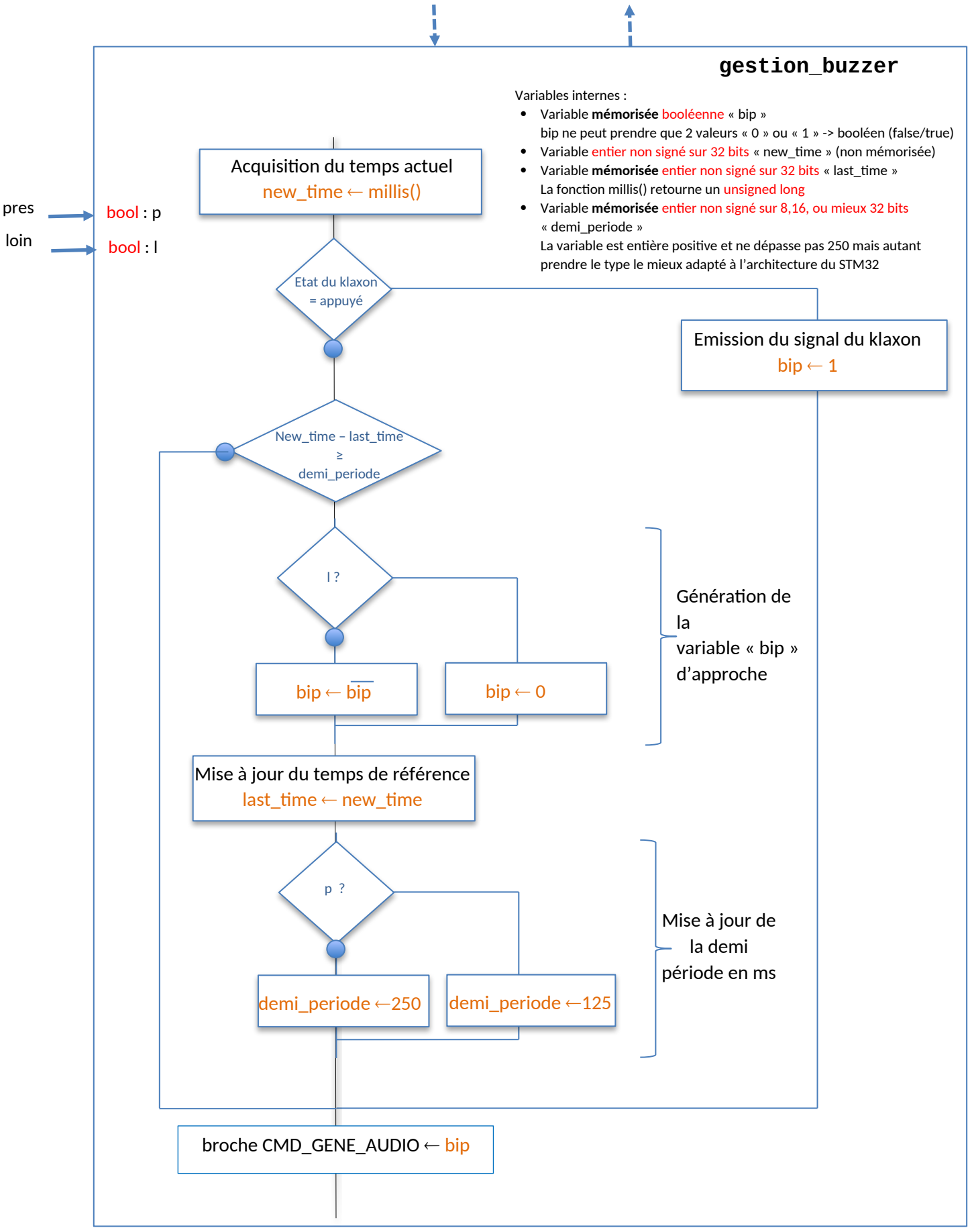
Afin d'éviter des bruits parasites, désagréables, à la limite des zones ; les entrées (près, loin ...) ne seront prises en considération que toutes les demi-périodes (125 ms si zone près, 250 ms sinon).

Piloter la sortie CMD_GENE_AUDIO qui commande l'oscillateur de la carte shield avec la gestion du temps par la fonction **millis()**

- Initialiser la demi-période à 125 ms
- Gérer le temps avec la fonction `millis()`
- Toutes les demi-périodes si pas de klaxon :
  - Générer une variable « `bip` » (`bip=1` pour la commande d'un son ; `bip=0` pour la commande d'un silence) en fonction des paramètres d'entrée `p et l`
  - Mise à jour de la demi-période en fonction du paramètre d'entrée `p`
  - Génération du signal de commande du son avec la fonction `digitalWrite()`
- Si le klaxon est activé alors klaxonner (le klaxon est prioritaire sur les bips)

Etat du bouton poussoir du klaxon

Algorigramme de la fonction :



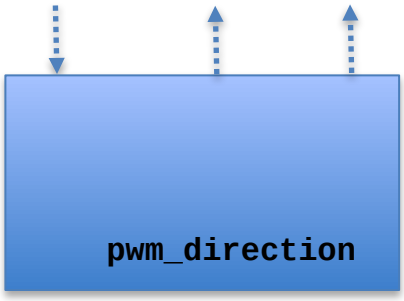
3) Fonction pwm_direction

Vue externe de la fonction :

Position du joystick sur l'axe X

Signal de commande PWM du moteur de direction à droite

Signal de commande PWM du moteur de direction à droite



Rôle de la fonction :

Cette fonction permet, à partir des informations de position du joystick sur l'axe X, de délivrer 2 signaux PWM.
Un signal PWM pour tourner à droite et un autre pour tourner à gauche.
Les signaux PWM doivent être proportionnels à la position du joystick.

Remarques :

Cette fonction est pilotée par la valeur que renvoie le joystick sur l'axe X

(-100 ≤ valeur de l'axe X ≤ 100)

La sortie PWM_DROITE commande la direction à droite
La sortie PWM_GAUCHE commande la direction à gauche

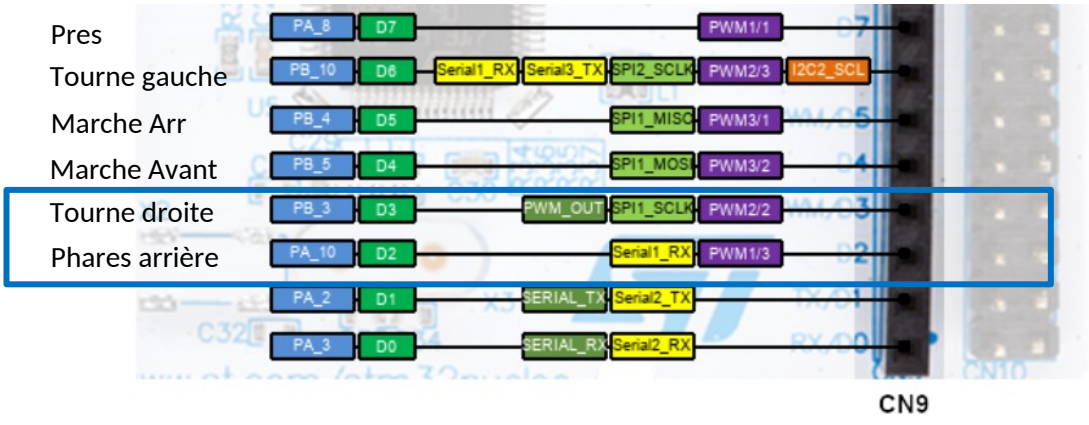
PWM_DROITE et PWM_GAUCHE ont une valeur comprise entre 0 et 100 (0 la roue est dans l'axe de la voiture, 100 la roue est tournée au maximum.

Afin de minimiser la consommation des moteurs, seules les consignes supérieures à 10 ou inférieures à -10 seront prises en compte.

Calcul et apport d'information :

La fréquence PWM direction imposé **19512.2 Hz**

D'après la documentation de la carte NUCLEO-L476RG :



Algorithme ou algorigramme de la fonction :

```
vide fonction pwm_direction(vide)
  uint_8 joystick_x ;
  joystick_x <- RemoteXY.joystick_01_x ;
  SI (RemoteXY.connect_flag == 1) ALORS
    SI (joystick_x > 10) ALORS
      timerDirection(CANAL_DROITE) <- joystick_x
      timerDirection(CANAL_GAUCHE) <- 0
    SINON SI (joystick_x < -10) ALORS
      timerDirection(CANAL_DROITE) <- 0
      timerDirection(CANAL_GAUCHE) <- (-1) * joystick_x
    SINON
      timerDirection(CANAL_DROITE) <- 0
      timerDirection(CANAL_GAUCHE) <- 0
    FinSI
  FinSI
FIN FONCTION
```

Type int8_t : RemoteXY encode la position du joystick entre -100 et +100. Le type int8_t (plage -128 à +127) est donc suffisant et cohérent avec le type interne de la bibliothèque.

Zone morte ±10 : Le joystick mécanique ne revient jamais exactement à 0 quand il est relâché. Sans zone morte, les servos reçoivent des micro-commandes parasites en permanence. Le seuil ±10 représente 10% de la plage totale (100 unités), ce qui absorbe le bruit sans dégrader la réactivité.

(-1) × joystick_x pour le canal gauche : Quand le joystick est dévié à gauche, joystick_x est négatif (ex : -75). Or `setCaptureCompare()` attend un rapport cyclique positif. On multiplie par -1 pour obtenir la valeur absolue : (-1) × (-75) = 75, soit 75% de rapport cyclique sur le servo gauche.

Plage 0 à 100 de `setCaptureCompare()` : Le timer est configuré avec `setOverflow(100)`, donc le compteur va de 0 à 100. La valeur passée correspond directement au rapport cyclique en % : 0 = arrêt, 100 = pleine vitesse. La valeur du joystick (0 à 100) s'utilise donc sans conversion.

connect_flag == 1 : Cette condition vérifie que le smartphone est connecté en Bluetooth avant d'envoyer toute commande. Si la connexion est perdue, aucune commande n'est envoyée aux servos, évitant tout comportement imprévisible de la voiture.

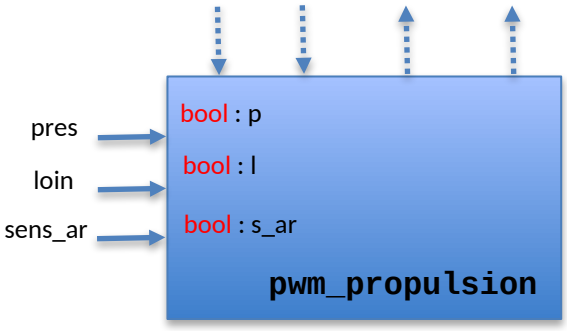
4) Fonction `pwm_propulsion`

Vue externe de la fonction :



Rôle de la fonction :

Cette fonction permet, à partir des informations de position du joystick sur l'axe Y, des informations `pres` et `loin` du capteur arrière et de la vitesse d'approche de l'obstacle à l'arrière, de délivrer 2 signaux PWM et de retourner une variable `freinage` qui servira dans une autre fonction pour allumer les feux arrière.
La consigne de vitesse est réduite de moitié dans la zone "moyenne distance" pour minimiser la distance de freinage dans la zone "près".
La consigne de vitesse est forcée à 0 dans la zone "près".
Le freinage est maintenu tant que le véhicule se rapproche de l'obstacle dans la zone "près" uniquement (consigne de vitesse maximale en sens inverse durant le freinage jusqu'à arrêt du véhicule).



Deux signaux PWM sont à générer : un pour la marche avant et un autre pour la marche arrière.

En cas de perte du signal bluetooth la consigne de vitesse est forcée à 0 afin d'empêcher la destruction du véhicule.

Remarques :
Cette fonction prend en compte les informations de zone `pres` et `loin` du capteur arrière ainsi que l'information `sens_ar` qui prévient d'un rapprochement avec un obstacle en marche arrière.
2 autres champs sont utilisés à partir de la structure globale `RemoteXY` comme :

- La valeur que renvoie le joystick sur l'axe y
  (-100 ≤ valeur de l'axe Y ≤ 100)
- L'indicateur de non-perte de liaison Bluetooth

La sortie PWM_AVANT commande la propulsion avant si PWM_ARRIERE = **2**
La sortie PWM_ARRIERE commande la propulsion arrière si PWM_AVANT = **3**

Les rapports cycliques des sorties PWM_AVANT et PWM_ARRIERE ont une valeur comprise entre 0 et 100 (0 les roues sont à l'arrêt et 100 les roues tournent à vitesse maximale)
Afin de minimiser la consommation des moteurs, seules les consignes supérieures à 10 ou inférieures à -10 seront prises en compte.

Informations et calculs :

Overflow des PWM est pris à 100 pour des raisons de simplicité avec le pourcentage

Les calculs du PRESCALER sont les mêmes que pour la direction (voir la fonction précédente)

Le timer de propulsion est le timer ?

Le canal pour la marche avant est le canal ?

Le canal pour la marche arrière est le canal ?

sens_ar = 0 si l'obstacle se rapproche de l'arrière

Algorithme ou algorigramme de la fonction :

```
booléen fonction pwm_propulsion(booléen : p, booléen : l, booléen : s_ar)

  booléen freiner <- FAUX
  SI (connect_flag == 1) ALORS                // Smartphone connecté
    SI (joystick_y > 10) ALORS                // Marche avant
      timerPropulsion(CANAL_AVANT)   <- joystick_y
      timerPropulsion(CANAL_ARRIERE) <- 0
    SINON SI (joystick_y < -10) ALORS         // Demande marche arrière
      SI (l == VRAI) ALORS                    // Loin : marche arrière pleine
        timerPropulsion(CANAL_AVANT)   <- 0
        timerPropulsion(CANAL_ARRIERE) <- joystick_y × (-1)
      SINON SI (p == FAUX) ALORS              // Moyen : marche arrière réduite
        timerPropulsion(CANAL_AVANT)   <- 0
        timerPropulsion(CANAL_ARRIERE) <- (joystick_y × (-1)) / 2
      SINON                                   // Près : obstacle à l'arrière
        SI (s_ar == FAUX) ALORS               // Voiture s'approche = freinage
          timerPropulsion(CANAL_AVANT)   <- 100
          timerPropulsion(CANAL_ARRIERE) <- 0
          freiner ← VRAI
        SINON                                 // Voiture s'éloigne = arrêt
          timerPropulsion(CANAL_AVANT)   <- 0
          timerPropulsion(CANAL_ARRIERE) <- 0
        FINSI
      FINSI
    SINON                                     // Joystick centré
      SI (p == VRAI) ALORS                    // Obstacle proche = freinage
        SI (s_ar == FAUX) ALORS               // Voiture s'approche = freinage
          timerPropulsion(CANAL_AVANT)   <- 100
          timerPropulsion(CANAL_ARRIERE) <-0
          freiner ← VRAI
        SINON                                 // Voiture s'éloigne = arrêt
          timerPropulsion(CANAL_AVANT)   <- 0
          timerPropulsion(CANAL_ARRIERE) <- 0
        FINSI
      SINON                                   // Pas d'obstacle = arrêt
        timerPropulsion(CANAL_AVANT)   <- 0
        timerPropulsion(CANAL_ARRIERE) <- 0
      FINSI
    FINSI
  FINSI
  RETOURNER freiner
FIN FONCTION
```

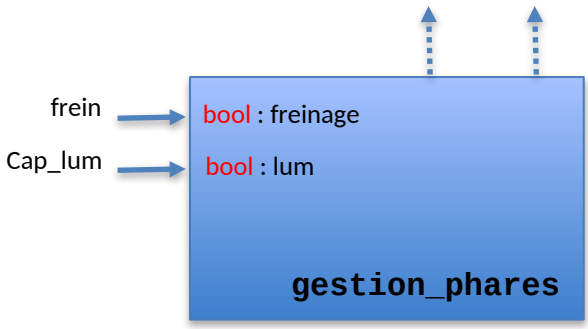
5) Fonction gestion phares

Rôle de la fonction :

Dans cette fonction, les phares arrière s'allument à 30% (PWM) lorsque les phares avant sont allumés et à 100% si freinage même si les phares avant sont éteints. L'indication de freinage est fournie par la fonction propulsion.

Les phares s'allument automatiquement en cas de faible luminosité. Le seuil entre nuit et jour est de 1.5V (revoir le chapitre de capteur de luminosité de la partie électronique.

Vue externe de la fonction :



Informations et calculs :

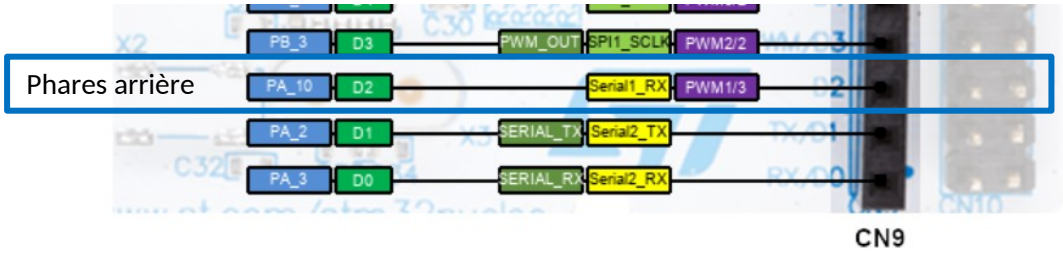
La fréquence personnalisée du signal PWM phares arrière

Overflow des PWM est pris à 100 pour des raisons de simplicité avec le pourcentage

Les calculs du PRESCALER sont les mêmes que pour la direction (voir la fonction précédente)

Le timer de propulsion est le timer ? **1**

Le canal pour phares arrière est le canal ? **3**



La limite jour/nuit pour Cap_lum correspond à 1.5V soit une valeur numérique de …….

Cap_lum <= ……. : jour

Cap_lum > ……. : nuit

Algorithme ou algorigramme de la fonction :

```
vide fonction gestion_phares(booléen : freinage, uint32_t : lum)
   booléen phareAv <- FAUX
   SI (lum <= 1861) ALORS                    // Mode jour (clair)
    SI (freinage == VRAI) ALORS             // Freinage = phares AR à 100%
   timerPhares(CANAL_PHARE) <- 100
       SINON                            // Pas de freinage = phares AR éteints
         timerPhares(CANAL_PHARE) <- 0
       FINSI
     SINON SI (lum > 1861) ALORS              // Mode nuit (sombre)
       phareAv ← VRAI                         // Allumer les phares avant
       SI (freinage == VRAI) ALORS         // Freinage = phares AR à 100%
         timerPhares(CANAL_PHARE) <- 100
       SINON                               // Pas de freinage = phares AR à 30%
         timerPhares(CANAL_PHARE) <- 30
       FINSI
     FINSI
    SI (phareAv == VRAI) ALORS               // Phares avant
       PHARES_AV_PIN <- HIGH
     SINON
       PHARES_AV_PIN <- LOW
     FINSI
   FIN FONCTION
```

6) Fonction gestion température

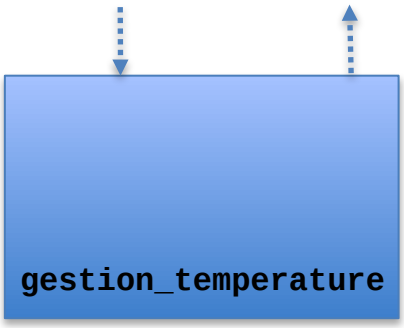
Vue externe de la fonction :

Liaison I2C

Cap temp

RemoteXY

température



Rôle de la fonction :

- Cette fonction permet un affichage de la température extérieure sur le SMARTPHONE.
- L'adresse du capteur est personnalisée (voir les 3 cavaliers sur la carte pour l'adresse personnalisée)
- Une amélioration de la fonction est que la température ne soit rafraichie que toutes les 3 secondes.

Compléments d'information :

L'adresse du capteur de température I2C est de la forme 01001XXX

Avec **011** les 3 bits personnalisées (monôme 5)

7) Fontion gestion capteur avant

Rôle de la fonction :

Cette fonction permet de faire l'acquisition de la distance de l'obstacle à l'avant.
Pour ne pas être perturbé par les parasites électriques et obtenir une valeur suffisamment stable cette fonction fait l'acquisition de la valeur du capteur toutes les 10 ms et la range dans un tableau de 10 valeurs avec un indice (index) glissant, puis fait la moyenne sur les 10 valeurs du tableau et retourne cette valeur qui sera utilisée par ailleurs.

Vue externe de la fonction :



Algorithme ou algorigramme de la fonction :

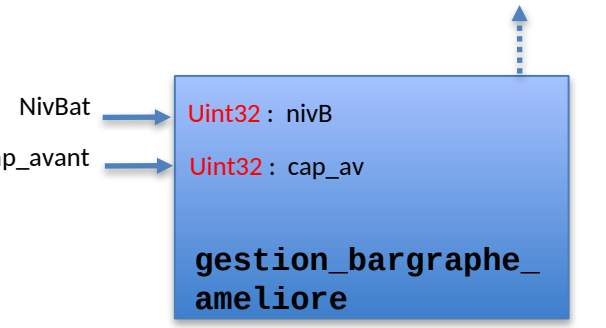
```
Uint32 fonction gestion_capteur_av (uint32 capt_av)
  // Variables statiques (conservées entre les appels)
  Uint32 static tempPrecedent <- 0
  Uint32 static tab[10] <- {0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
  Uint32 static indextab <- 0
  Uint32 tempActuel <- millis()
  Uint32 static somme <- 0
  // Acquisition toutes les 10 ms
  SI (tempActuel - tempPrecedent >= 10) ALORS
    tempPrecedent <- tempActuel
    tab[indextab] <- capt_av        // Stockage valeur courante
    indextab <- indextab + 1
    SI (indextab >= 10) ALORS         // Index glissant : retour à 0
      indextab <- 0
    FINSI
  FINSI
  // Calcul de la moyenne sur les 10 valeurs
  POUR indextab_boule DE 0 À 9 FAIRE
    somme <- somme + tab[indextab_boule]
  FPOUR
  RETOURNER somme / 10
FIN FONCTION
```

8) Fonction gestion bargraphe améliorée

Vue externe de la fonction :

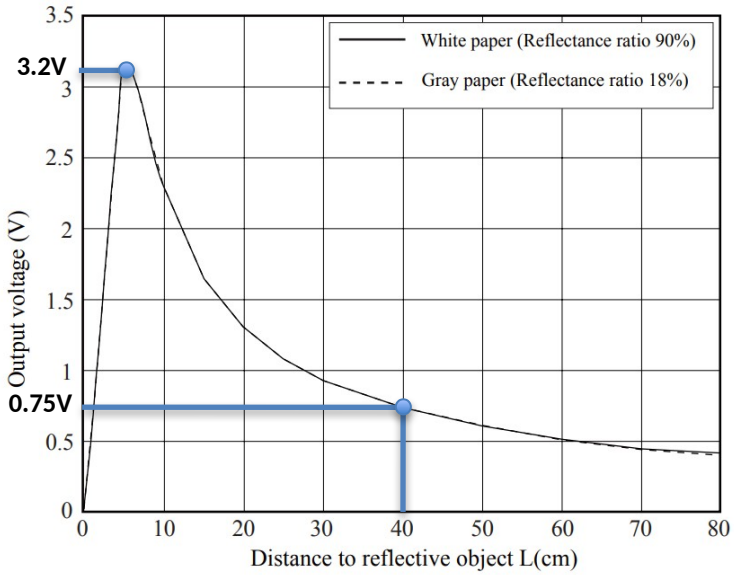
Rôle de la fonction :

Cette fonction reprend le fonctionnement du bargraphe du début. La proximité d'un obstacle avec le capteur avant est indiqué sur le bargraphe à la place du niveau de batterie lorsque la voiture est dans la zone "près".
Dès que la voiture sort de la zone "près", à l'avant, le bargraphe reprend son fonctionnement normal de l'affichage du niveau de batterie.

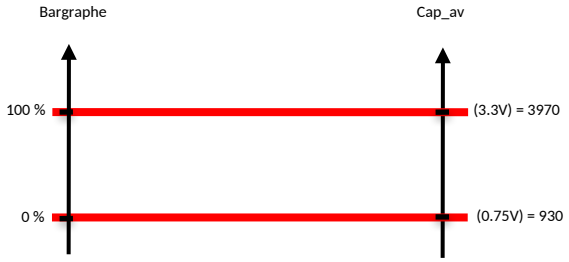


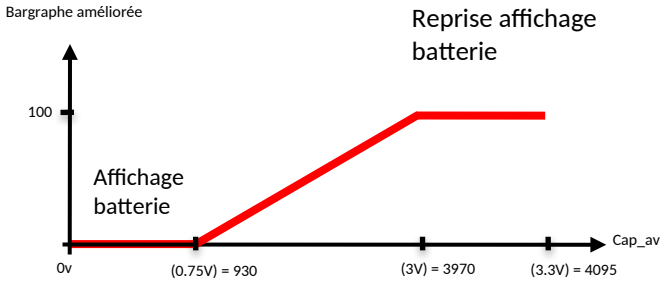
Calculs et dimensionnement

Courbe du capteur de distance avant en fonction de la distance



Algorithme ou algorigramme de la fonction :





Pour trouver l'équation d'une droite passant par deux points :

Equation de droite :

1. Calculer le coefficient directeur ()

2. Calculer l'ordonnée à l'origine ()

3. L'équation finale

9) Gestion sens avant

Rôle de la fonction :

Cette fonction utilise la valeur moyenne retournée par la fonction `gest_capt_avant` pour retourner, elle-même, une valeur indiquant une approche d'obstacle à l'avant.
Toutes les 100 ms une comparaison est effectuée. Si la valeur moyenne de la position a augmenté de plus de 5 alors le signal d'approche retourné par la fonction vaut `true` sinon `false`.

D. Mise en œuvre

1) Fonction `gestion_bargraphe`

Code de l'initialisation du CAN à 12 bits : `analogReadResolution(12);`

Code de la déclaration de la variable niv_bat dans le loop : `int nivBat= analogRead(NIV_BATT_PIN);`

Code de la lecture de la tension niveau batterie (« niv_bat ») :
(Dans la partie acquisition de l'environnement)

Traduction en C++ de la fonction (copie d'écran) du code :

```
void gestion_bargraphe(uint32_t nivB) {
    // Remplace x et y par les valeurs ou variables réelles
    const uint32_t x = 3102;  // seuil à 2.5V = (4095*2.5)/3.3
    const uint32_t y = 3723;  // seuil à 3V = (4095*3)/3.3
    // uint16_t a;
    if (nivB <= x) {
        RemoteXY.Bargraph = 0;  // Barre vide
    }
    else if (nivB >= y) {
        RemoteXY.Bargraph = 100;  // Barre plein
    }
    else
    {
      RemoteXY.Bargraph = 0.16117 * nivB - 500;  // Barre entre 2.5V et 3V

    }
}
```

Code de la boucle infinie mettant en œuvre la fonction :

```
gestion_bargraphe(nivBat);
```

2) Fonction `gestion_buzzer`

Traduction en C++ (copie d'écran du code) :

```
void gestion_buzzer(bool p, bool l)
{
  static uint32_t tempsPrecedent = millis();
  static bool buzzerState = LOW;
  static uint32_t demiPeriod = 250;
  uint32_t tempsActuel = millis();

  // Klaxon du smartphone (prioritaire sur le reste)
  if (RemoteXY.klaxon == 1) // On peux également utiliser juste klaxon avec
(RemoteXY.klaxon = klaxon)
  {
    buzzerState = HIGH;
  }
  // Gestion des bips
  else
  {
    // 4 bips par seconde lorsque la voiture est dans la zone près à l'avant ou à
l'arrière
    if (p == true)
    {
      demiPeriod = 125; // 125ms = 4 bips/sec
    }
    else
    {
      demiPeriod = 250; // 250ms = 2 bips/sec
    }
    if (tempsActuel - tempsPrecedent >= demiPeriod)
    {
      tempsPrecedent = tempsActuel;

      // 2 bips par seconde lorsque la voiture est dans la zone moyen à l'avant ou à
l'arrière
      if (l == false)
      {
        buzzerState = !buzzerState;
      }
      else
      {
        buzzerState = LOW;
      }
    }
  }
  digitalWrite(CMD_GENE_AUDIO_PIN, buzzerState);
}
```

Code de la boucle infinie mettant en œuvre la fonction :

```
  gestion_buzzer(pres, loin);
```

3) Fonction pwm_direction

Déclaration des canaux :

```
// Déclaration des canaux utilisés pour les PWM  //
const uint32_t CANAL_DROITE = 2;
const uint32_t CANAL_GAUCHE = 3;
```

Initialisation du timer et des canaux dans le setup :

```
timerDirection.setPrescaleFactor(44);
timerDirection.setOverflow(100);
timerDirection.setMode(CANAL_DROITE,
TIMER_OUTPUT_COMPARE_PWM1, PWM_DROITE_PIN);
timerDirection.setMode(CANAL_GAUCHE,
TIMER_OUTPUT_COMPARE_PWM1, PWM_GAUCHE_PIN);

timerDirection.setCaptureCompare(CANAL_DROITE, 0);
timerDirection.setCaptureCompare(CANAL_GAUCHE, 0);

timerDirection.resume();
```

Déclaration du timer :

```
// Déclaration des timers            //
HardwareTimer timerDirection(TIM2);
HardwareTimer timerPropulsion(TIM2);
```

Traduction en C++ (copie d'écran du code) :

```
void pwm_direction() {
    if (joystickX > 10) {
        PWM_DROITE = joystickX;
        PWM_GAUCHE = 0;
    }
    else if (joystickX < -10) {
        PWM_DROITE = 0;
        PWM_GAUCHE = abs(joystickX);
    }
    else {
        PWM_DROITE = 0;
        PWM_GAUCHE = 0;
    }
}
```

Code de la boucle infinie mettant en œuvre la fonction :

```
pwm_direction();
```

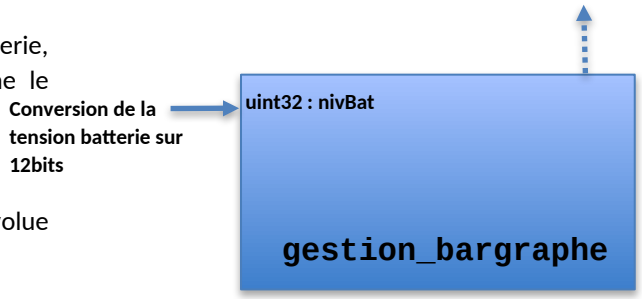
E. Tests unitaires

Niveau bargraphe

1) test unitaire de la fonction `gestion_bargraphe`

Rôle de la fonction :

Cette fonction permet, à partir de la tension batterie, d'indiquer sur un bargraphe à l'écran du smartphone le niveau de batterie.
Au-dessus de 3 V la batterie est pleine.
En dessous de 2.5 V la batterie est vide.
Entre 2.5 V et 3 V le pourcentage de la batterie évolue linéairement en fonction de la tension



Objectif du test :
Montrer que le niveau batterie du bargraph est bien en accord avec la conversion de la tension batterie.

Environnement du test

- La carte Shield est en place sur la carte microcontrôleur de la voiture
- Le module Bluetooth est en place sur la carte Shield
- Le connecteur d'alimentation est en place sur la carte Shield
- Le smartphone n'est pas connecté avec l'application RemoteXY
- L'environnement de débuggage est lancé
- **Pas de source de tension branchée directement sur la carte Nucleo (risque de destruction si dépassement de 3.3V)**

Jeu de tests
La conversion de la tension batterie est modifiée en débuggage et la valeur du niveau bargraphe est visualisée dans le débugger.

- Les variables espionnées sont **RemoteXY.Bargraph** et **nivB**
- La ou les variables modifiées sont **nivB**

Résultats attendus

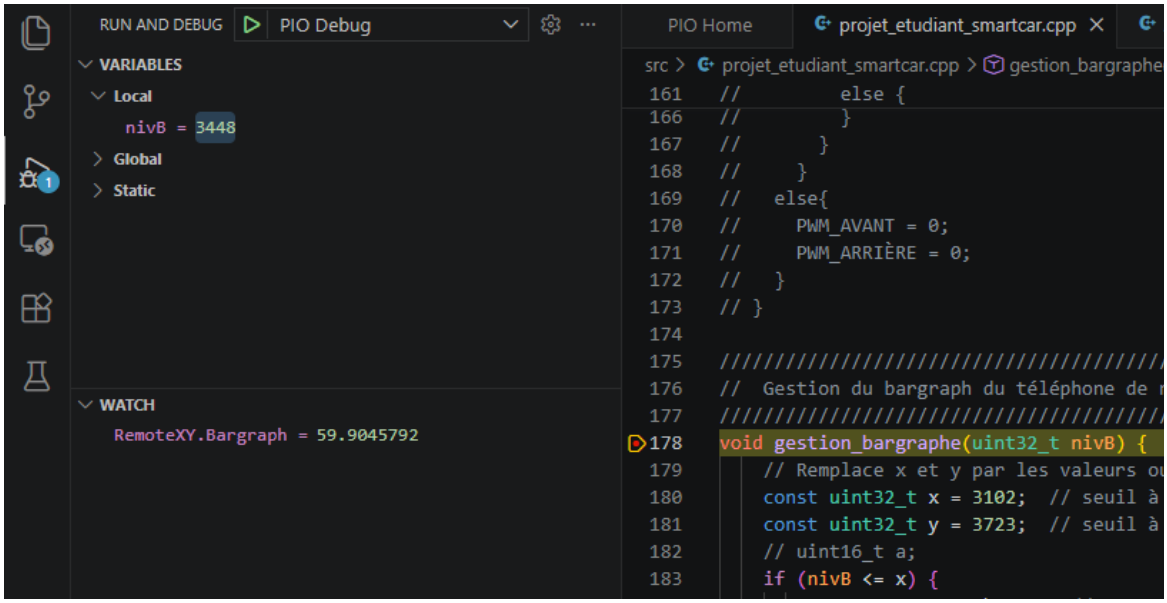
Indiquer la résolution de l'ADC choisie :

Tension batterie en Volt	Valeur numérique **espérée** de la conversion Tension batterie	Niveau **espéré** du bargraphe en %
< 2.5V	<3102	0 %
= 2.5V	3102	0 %
= 2.8V	3474	60 %
= 3V	3723	100 %
>3V	>3723	100 %

Résultats des tests

Tension batterie en Volt	Valeur numérique de la conversion Tension batterie	Niveau du bargraphe en %
< 2.5V	2000	0 %
= 2.5V	3102	0 %
= 2.8V	3448	59.9 %
= 3V	3723	100 %
>3V	4000	100 %

Copie de l'écran du débugueur pour VnivBat correspondant à 2.8V = **3448**



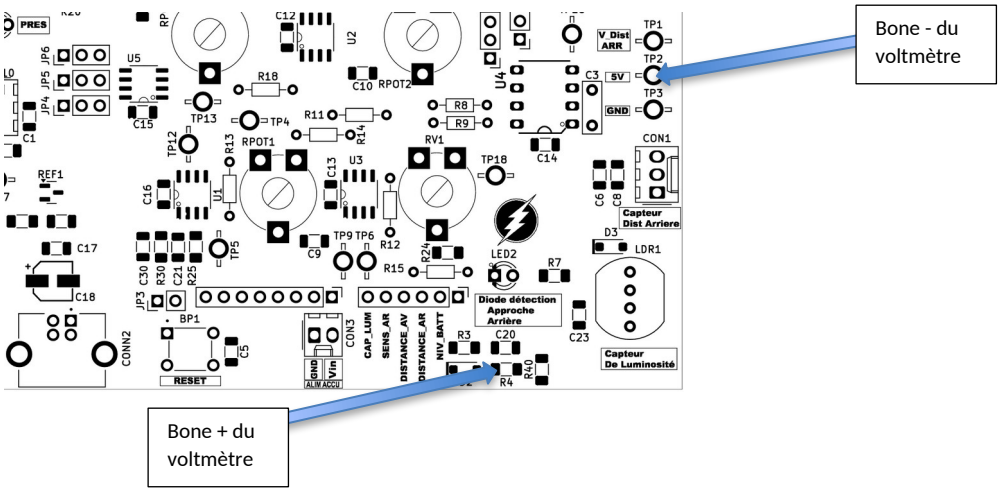
Conclusion :

Les résultats des jeux de test correspondent à ceux attendus, la fonction est validée.

Test d'intégration

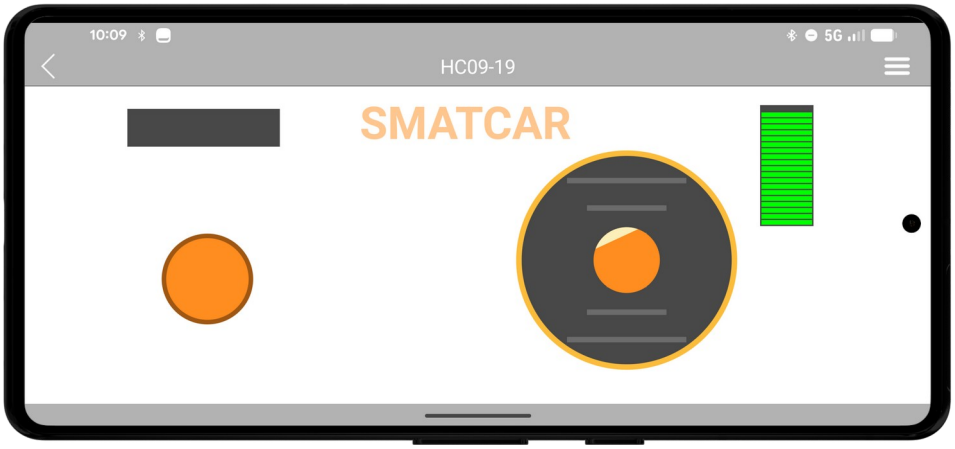
Objectif : Montrer que ce qui est affiché sur le téléphone est en accord avec la tension VnivBatt mesurée au voltmètre.

- Pour mesurer la tension Vniv_batt à l'aide voltmètre placer la borne (+) du multimètre à la résistance R4 et la borne (–) à la masse (GND) de TP2



Après mesure de la tension Vniv_batt au voltmètre, je relève une tension de **2.95V** ce qui fait environ 98 % de batterie par conversion de cette tension en pourcentage du bargraph

Copie d'écran du smartphone prouvant le bon fonctionnement du bargraph



Comparaison des pourcentages affichés et calculés : Le bargraph se situe entre deux seuils (95 < 98 < 100). La valeur mesurée étant proche de la limite supérieure, l'affichage oscille entre 19 et 20 barres, ce qui provoque le clignotement de la barre maximale.

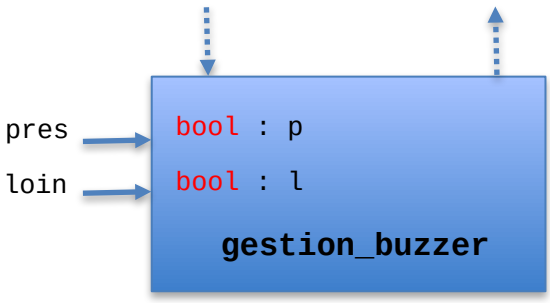
2) test unitaire de la fonction `gestion_buzzer`

Rôle de la fonction :

Cette fonction permet d'élaborer un signal de commande de l'oscillateur (`CMD_GENE_AUDIO`) afin d'émettre un signal sonore (bip) dans plusieurs cas de figure :

4) bip continu si appui sur le bouton Klaxon du smartphone (prioritaire sur le reste).
5) 2 bips par seconde lorsque la voiture est dans la zone moyen à l'avant ou à l'arrière.
6) 4 bips par seconde lorsque la voiture est dans la zone près à l'avant ou à l'arrière

Vue externe de la fonction :



Objectif des tests :

Montrer que les émissions sonores du buzzer répondent au cahier des charges (vérifier les timings et les priorités)

Environnement du test

- La carte Shield est en place sur la carte microcontrôleur de la voiture
- Le module Bluetooth est en place sur la carte Shield
- Le connecteur d'alimentation est en place sur la carte Shield
- Le smartphone n'est pas connecté avec l'application RemoteXY
- La ligne de code « `RemoteXY_Handler();` » est mise en commentaire pour ne pas faire de synchronisation avec le smartphone pendant le debug.
- Le programme est chargé sur la carte microcontrôleur
- L'environnement de débuggage n'est pas lancé car influence sur la fonction millis()
- Le connecteur du buzzer est en place sur la carte Shield
- Oscilloscope pour la mesure des temps du signal

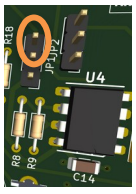
Ajouter ou supprimer des lignes suivant votre test

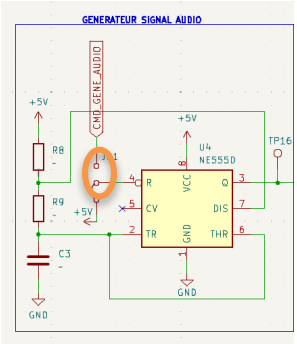
Jeu de tests

Les valeurs des variables `pres`, `loin` et et du champ `RemoteXY.klaxon` sont imposées dans le loop (Après les acquisitions pour écraser les valeurs de l'environnement). Ne pas oublier de Supprimer ces lignes de code après le test.

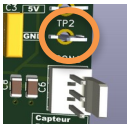
Utilisation de la maquette directement dans l'environnement avec visualisation du signal **Cmd_Gene_Audio** sur l'oscilloscope pour mesurer les temps et allure du signal

Borne+ de l'oscilloscope sur le cavalier de JP1, placé en position haute





Borne – de l'oscilloscope sur TP2 (GND de la carte)



Résultats **attendus**

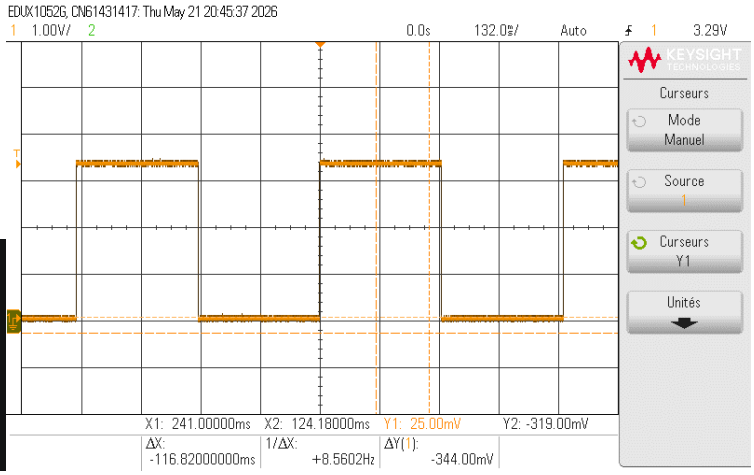
Loin	Pres	RemoteXY.klaxon		Cmd_Gene_Audio	Remarques
0	0	0		Signal carré de demi-période 250 ms	Obstacle entre 40 et 60 cm du capteur et sans klaxon => 2 bips par seconde
0	1	0		Signal carré de demi-période 250 ms	
1	0	0		0V	Sans obstacle à moins de 60 cm du capteur et sans klaxon => Pas de son

1	1	0		0 V	Cas improbable => Pas de son
0	0	1		3.3 V	L'appui sur le klaxon est prioritaire sur les bips
0	1	1		3.3V	L'appui sur le klaxon est prioritaire sur les bips
1	0	1		3.3V	L'appui sur le klaxon est prioritaire sur les bip continu
1	1	1		3.3 V	Obstacle entre 40 et 60 cm du capteur et sans klaxon => 2 bips par seconde

Résultats **effectifs** des tests

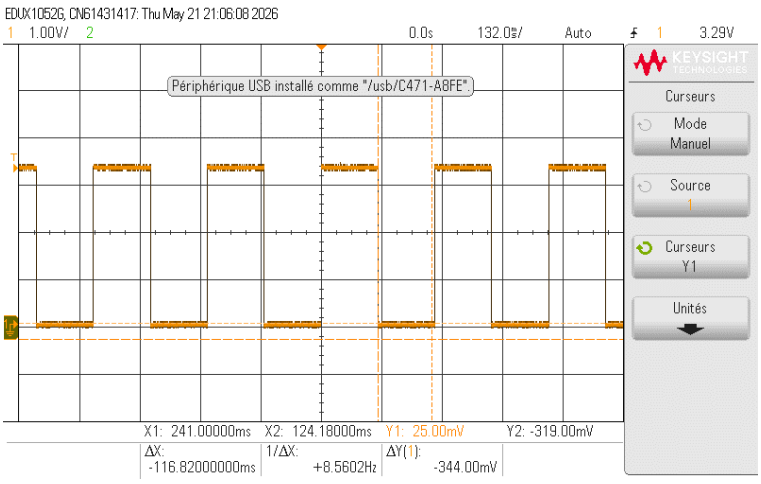
Loin	Pres	RemoteXY.klaxon		Cmd_Gene_Audio	Remarques
0	0	0		Signal carré de demi-période 250 ms	Obstacle entre 40 et 60 cm du capteur et sans klaxon => 2 bips par seconde

```
////////////////////////////////////
//  Jeu de test pour gestio_buzzer  //
////////////////////////////////////
pres = 0;
loin = 0;
RemoteXY.klaxon = 0;
```

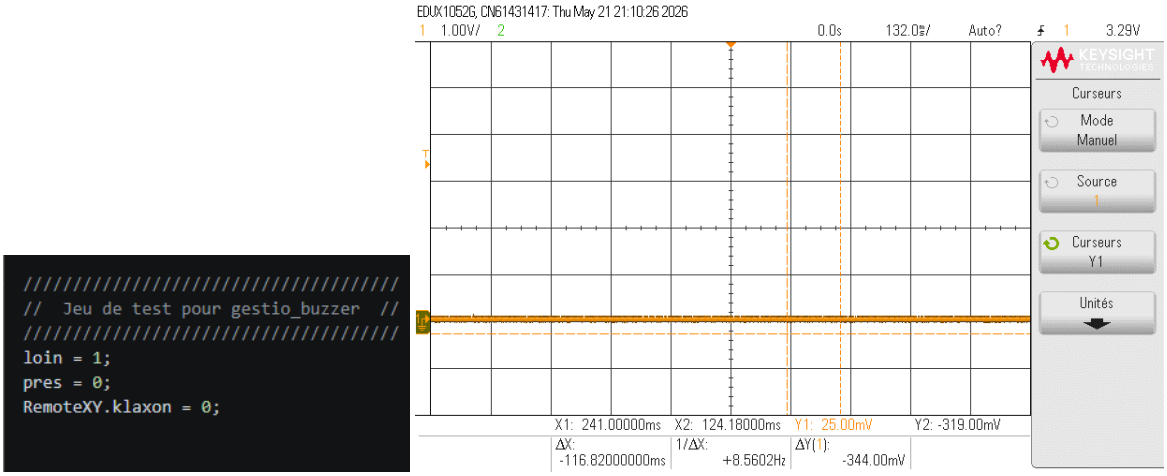


0	1	0			

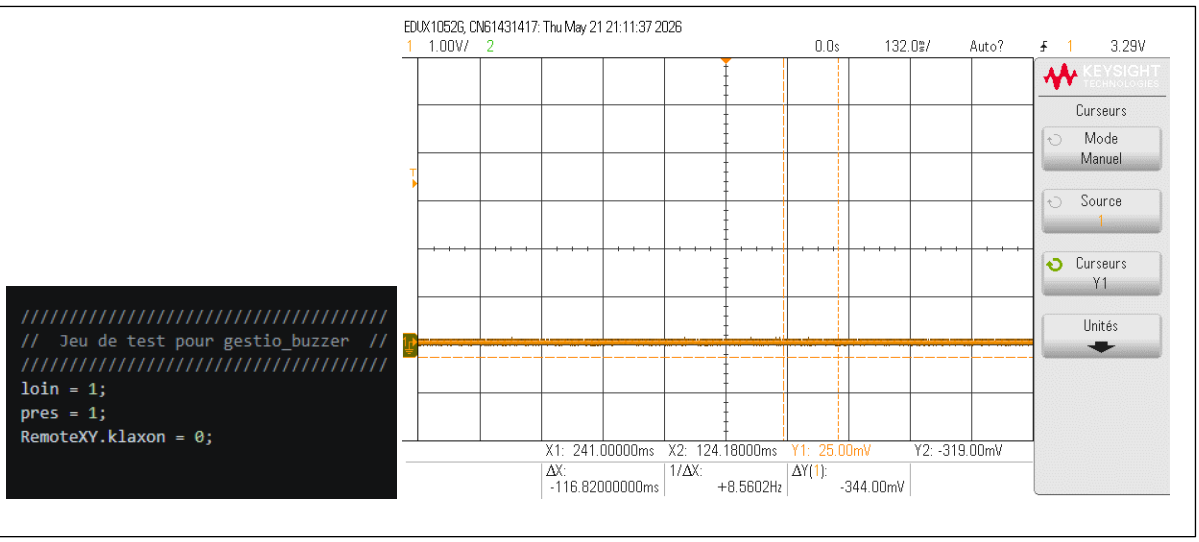
```
////////////////////////////////////
//  Jeu de test pour gestio_buzzer  //
////////////////////////////////////
loin = 0;
pres = 1;
RemoteXY.klaxon = 0;
```



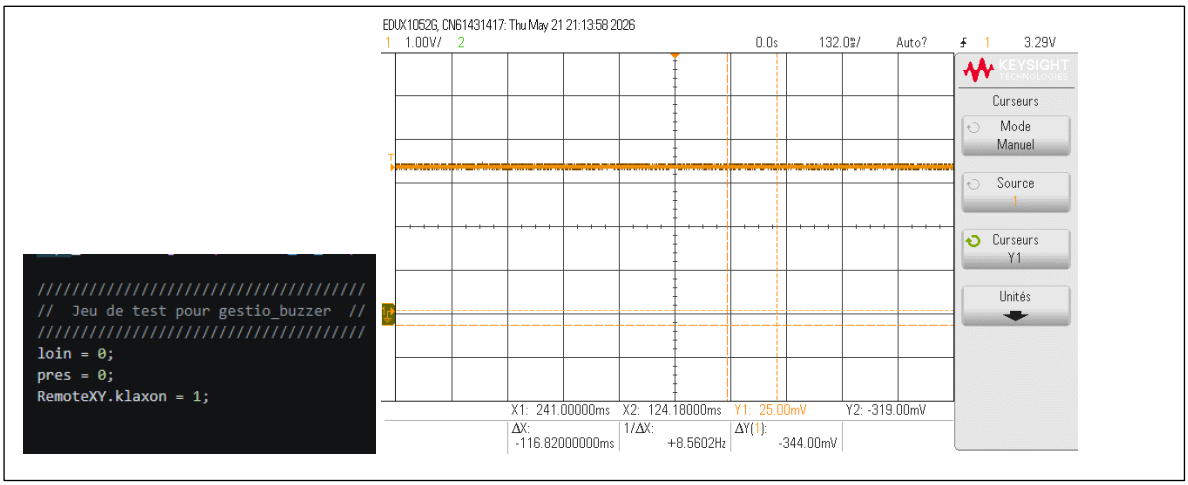
1	0	0		0V	Sans obstacle à moins de 60 cm du capteur et sans klaxon => Pas de son



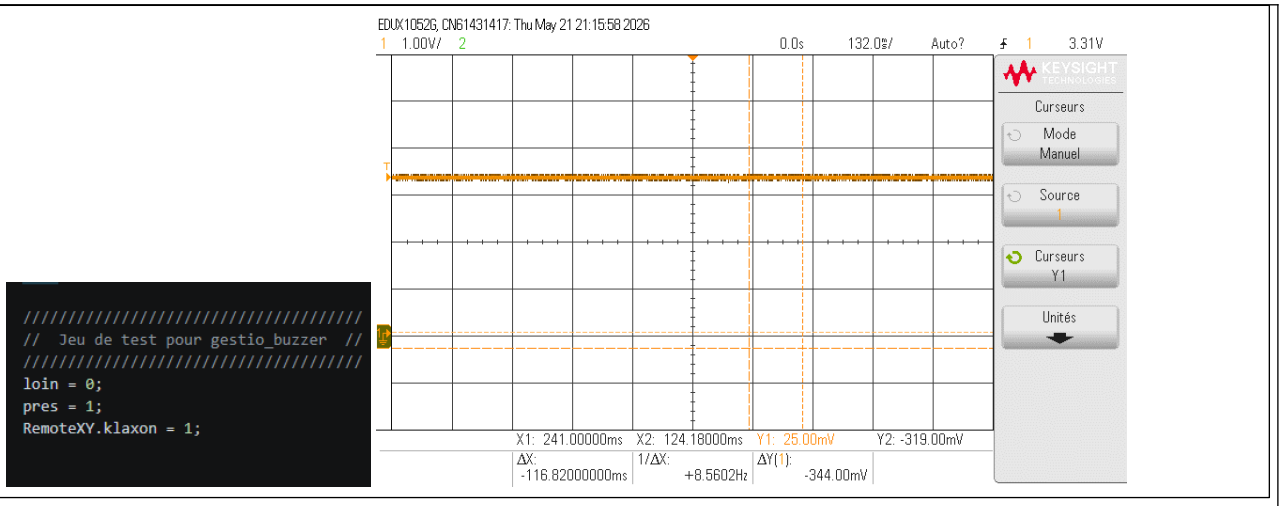
1	1	0		0V	Cas improbable => Pas de son



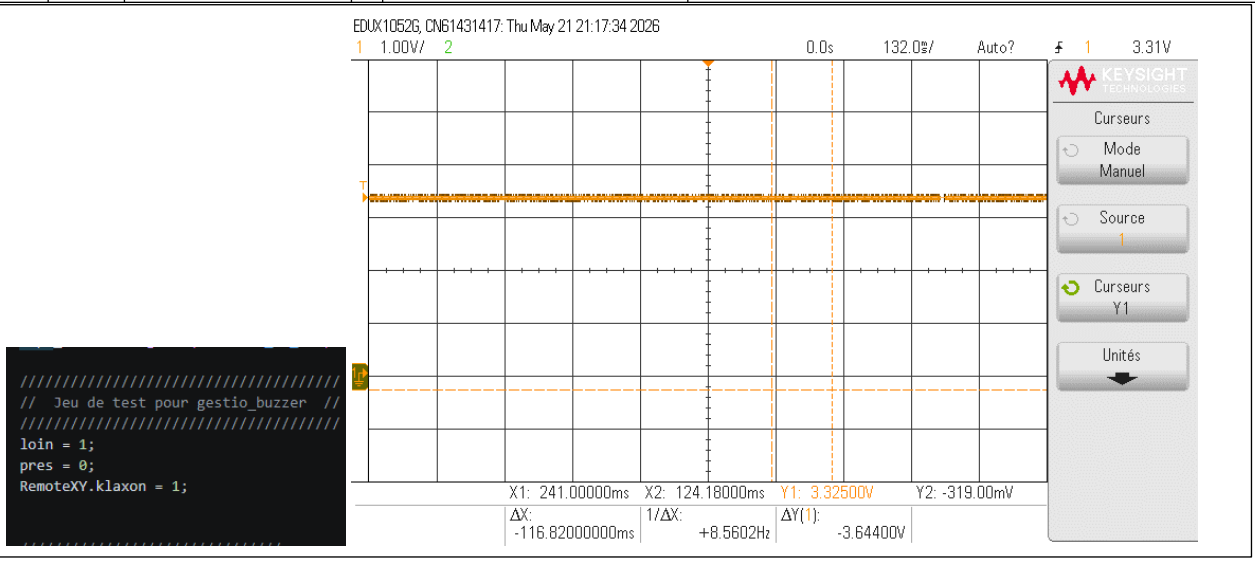
0	0	1		3.3V	L'appui sur le klaxon est prioritaire sur les bips



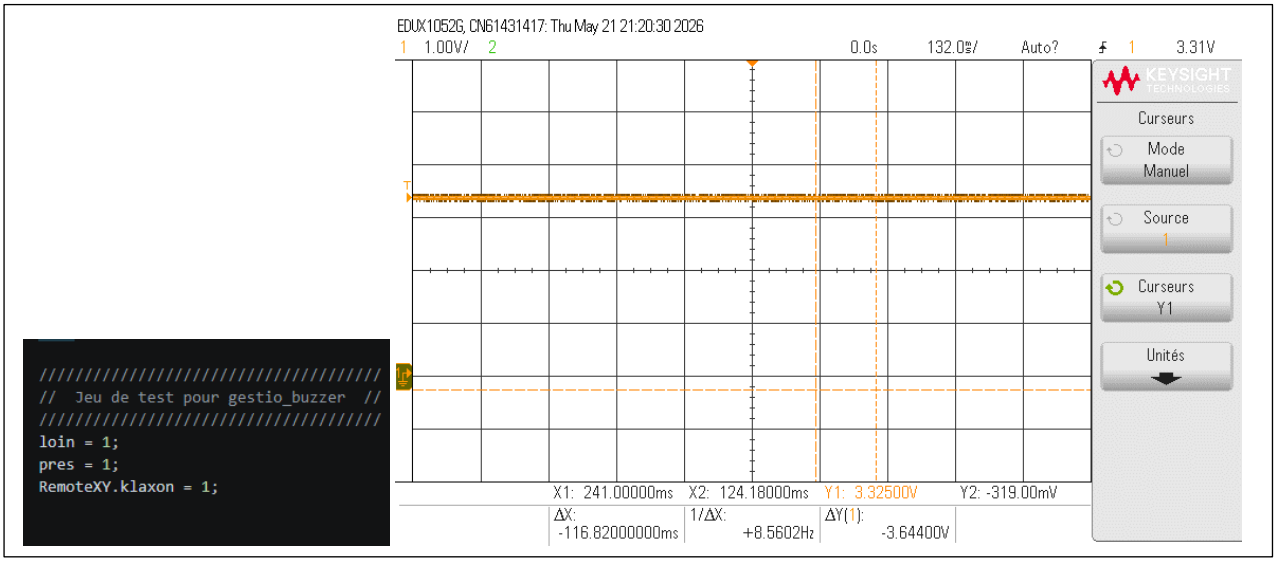
0	1	1		3.3V	L'appui sur le klaxon est prioritaire sur les bips



1	0	1		3.3V	L'appui sur le klaxon est prioritaire sur les bip continu



1	1	1		3.3V	Obstacle entre 40 et 60 cm du capteur et sans klaxon => 2 bips par seconde



Conclusion :

Les résultats des jeux de test correspondent à ceux attendus, la fonction est validée.

Démarche de correction si dysfonctionnement constaté :

Test d'intégration

Objectif : montrer que le comportement du buzzer répond au cahier des charges

Procéder aux tests de zone et de priorité du klaxon avec le son du buzzer, devant l'enseignant.

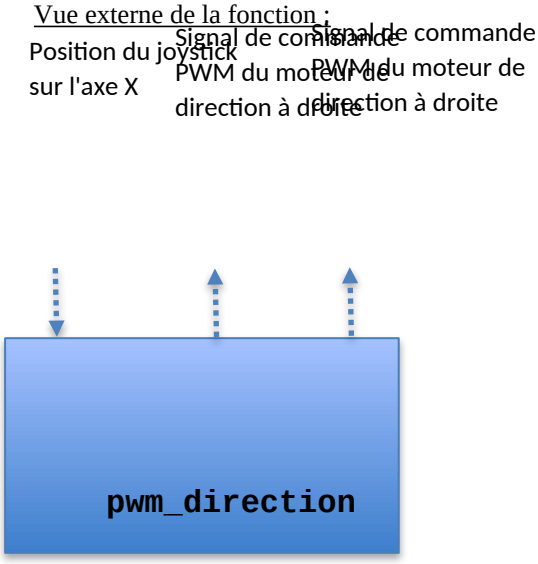
Klaxon	pres	loin		BUZZER
1	1	1		Bip continu, klaxon prioritaire
0	0	0		Pas de bip
0	0	1		Bip continue lent
0	1	0		Bip continue rapide

3) test unitaire de la fonction `pwm_direction`

Rôle de la fonction :

Cette fonction permet, à partir des informations de position du joystick sur l'axe X, de délivrer 2 signaux PWM
Un signal PWM pour tourner à droite et un autre pour tourner à gauche.
Les signaux PWM doivent être proportionnels à la position du joystick (sauf dans la zone "morte").

Vue externe de la fonction :



Objectif du test :
Montrer que les signaux PWM suivent les consignes du joystick sur l'axe X et vérifier la largeur de la zone morte.

Environnement du test

- La carte Shield est en place sur la carte microcontrôleur de la voiture.
- Le module Bluetooth est en place sur la carte Shield
- Le connecteur d'alimentation est en place sur la carte Shield.
- Le smartphone est connecté avec l'application RemoteXY.
- Le programme est chargé sur la carte microcontrôleur.
- L'environnement de débuggage **est** lancé.
- Les signaux PWM_DROITE et PWM_GAUCHE sont visualisés à l'écran de l'oscilloscope pour mesurer la fréquence et les rapports cycliques.

Jeux de test
La consigne RemoteXY sur l'axe X est directement modifiée dans le debuggeur :

- Clic droit sur la variable à espionner puis "add to watch"
- Clic droit sur la variable dans la fenêtre "watch" pour modifier la valeur

Le rapport cyclique et la fréquence des signaux PWM gauche et droite sont mesurés à l'oscilloscope.

La valeur du joystick sur l'axe X est visualisé dans le debuggeur.

Résultats attendus
Les roues tournent à gauche pour une consigne inférieure à -10, les roues tournent à droite pour une consigne supérieure à 10, et restent dans leur position initiale lorsque le joystick est entre -10 et 10 sur l'axe X.

La loi du rapport cyclique du signal PWM_droite en fonction de la consigne RemoteXY_joystickX doit être la suivante : **RemoteXY.joystick_01_x** pour une consigne du joystick > 10 sinon 0

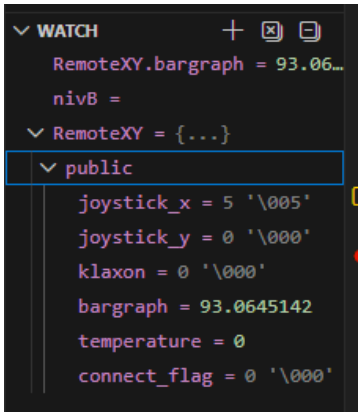
La loi du rapport cyclique du signal PWM gauche en fonction de la consigne RemoteXY_joystickX doit être la suivante : **- RemoteXY.joystick_01_x** pour une consigne du joystick < -10 sinon 0

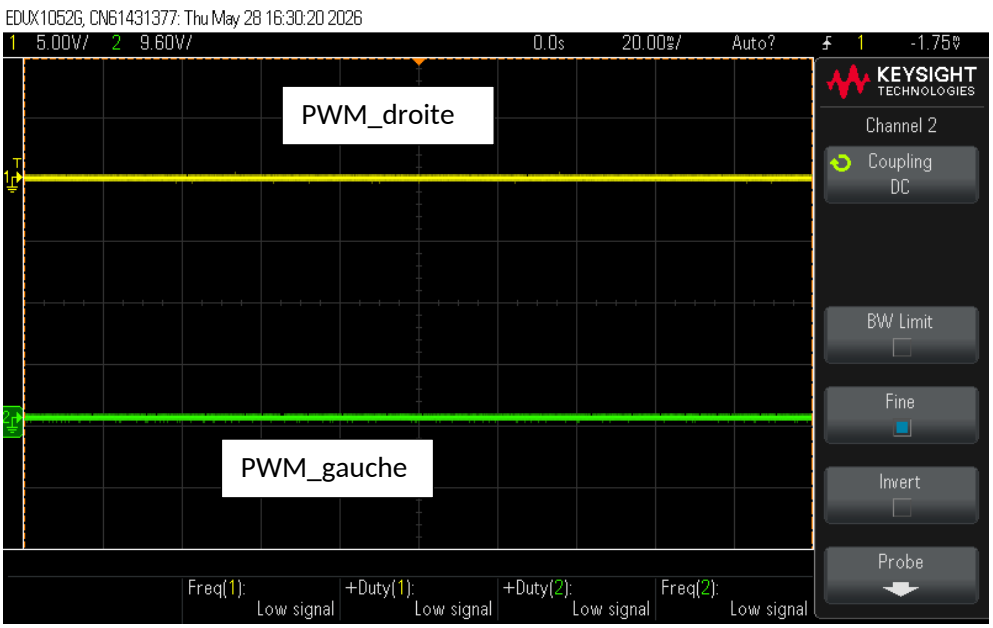
La fréquence des signaux PWM est de

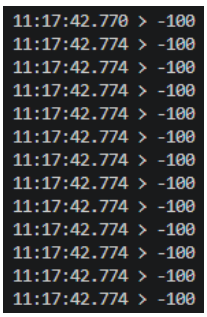
Avec

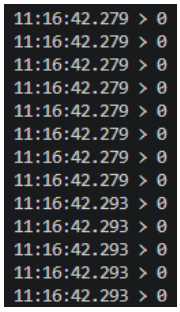
Résultats des tests

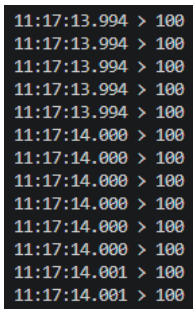
PWM_droite = **LOW** et PWM gauche = **LOW** lorsque RemoteXY_joystickX = **5**





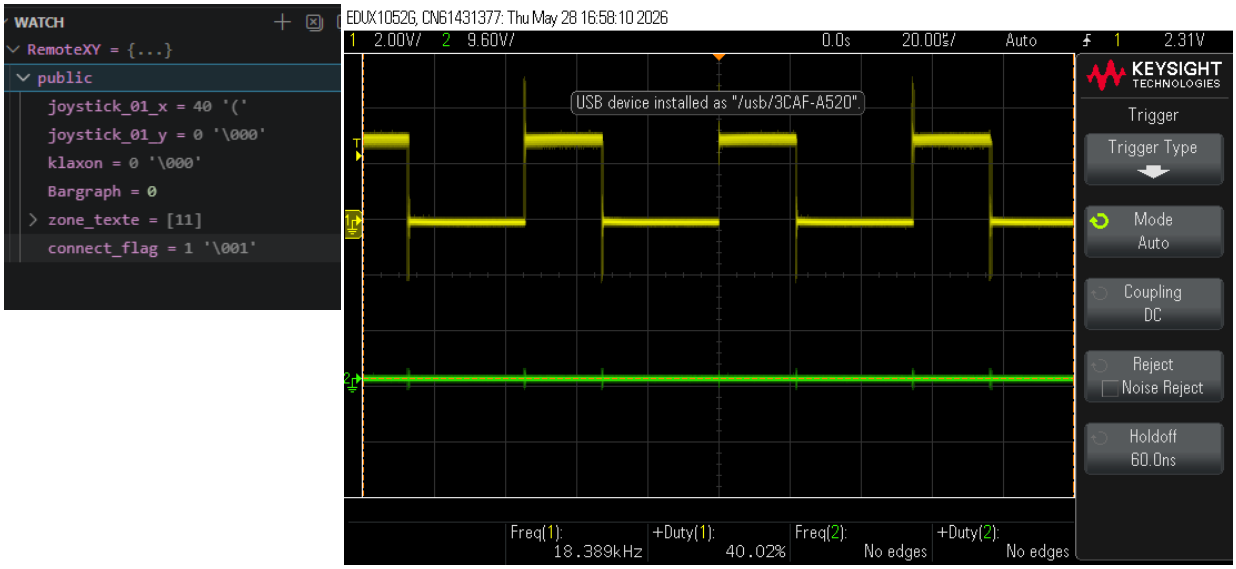




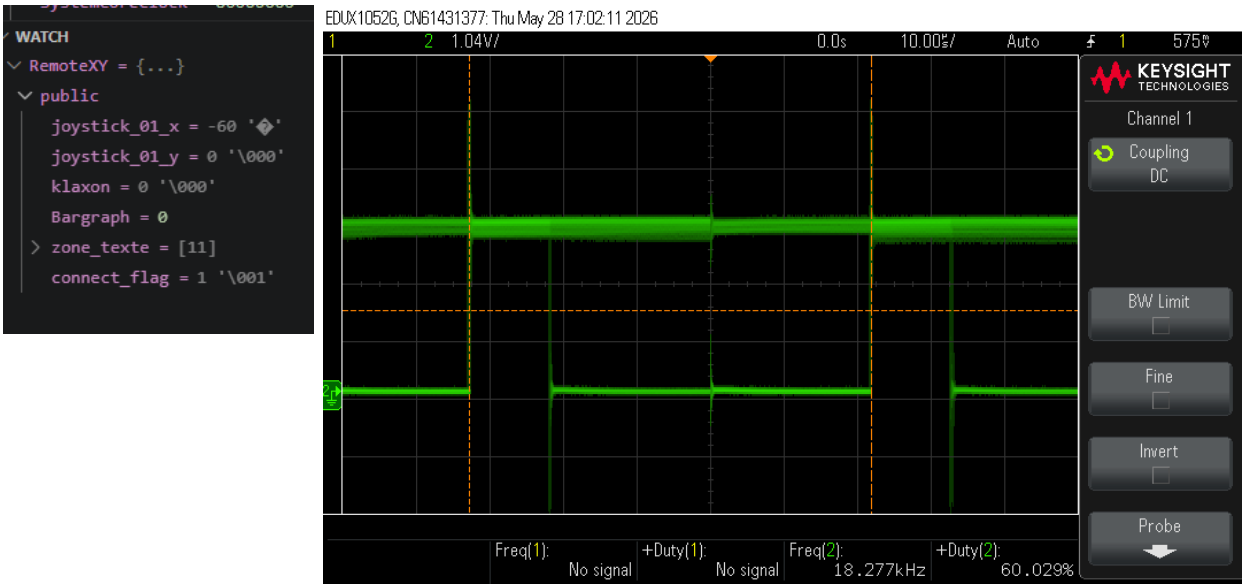


Débuggeur*

PWM_droite = 5 et PWM gauche = 0 lorsque RemoteXY_joystickX = 40 ou proche



PWM_droite = 5 et PWM gauche = 0 lorsque RemoteXY_joystickX = -60 ou proche



Conclusion :

Le test est concluant. Les relations entre rapport cyclique des PWM et la consigne du joystick sont exactes.
La fonction est validée.

Test d'intégration

Objectif : Montrer que les roues pivotent du bon côté en fonction de l'instruction donnée par l'utilisateur sur le joystick et vérifier l'existence de la zone morte.

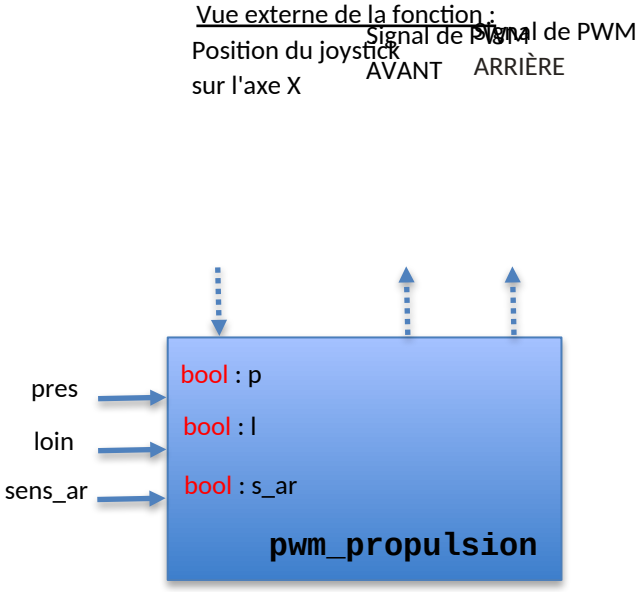
Le test est réussi selon les consignes du prof

4) test unitaire (Test d'intégration) de la fonction `pwm_propulsion`

Rôle de la fonction :

Cette fonction permet, à partir des informations de position du joystick sur l'axe Y, des informations `pres` et `loin` du capteur arrière et de la vitesse d'approche de l'obstacle à l'arrière, de délivrer 2 signaux PWM et de retourner une variable `freinage` qui servira dans une autre fonction pour allumer les feux arrière.
La consigne de vitesse est réduite de moitié dans la zone "moyenne distance" pour minimiser la distance de freinage dans la zone "près".
La consigne de vitesse est forcée à 0 dans la zone "près".
Le freinage est maintenu tant que le véhicule se rapproche de l'obstacle dans la zone "près" uniquement (consigne de vitesse maximale en sens inverse durant le freinage jusqu'à arrêt du véhicule).

Vue externe de la fonction :



Objectif du test :
Montrer que les signaux PWM suivent les consignent du joystick sur l'axe Y en fonction des zones près et loin.
Montrer le bon fonctionnement de l'anticollision

Environnement du test

- La carte Shield est en place sur la carte microcontrôleur de la voiture
- Le module Bluetooth est en place sur la carte Shield
- Le connecteur d'alimentation est en place sur la carte Shield
- Le programme est chargé sur la carte microcontrôleur
- L'environnement de débuggage **est** lancé
- Les variables sont espionnées et modifiées dans le débuggeur
- Les signaux PWM avant et arrière sont visualisés à l'écran de l'oscilloscope pour mesurer la fréquence et les rapports cycliques

Jeux de test

La consigne RemoteXY sur l'axe Y, la variable de connection bluetooth "connect_flag" les variables "pres", "loin", "sens_ar" sont directement modifiées avant l'appel de la fonction, dans le debuggeur :

- Clic droit sur la variable à espionner puis "add to watch"
- Clic droit sur la variable dans la fenêtre "watch" pour modifier la valeur

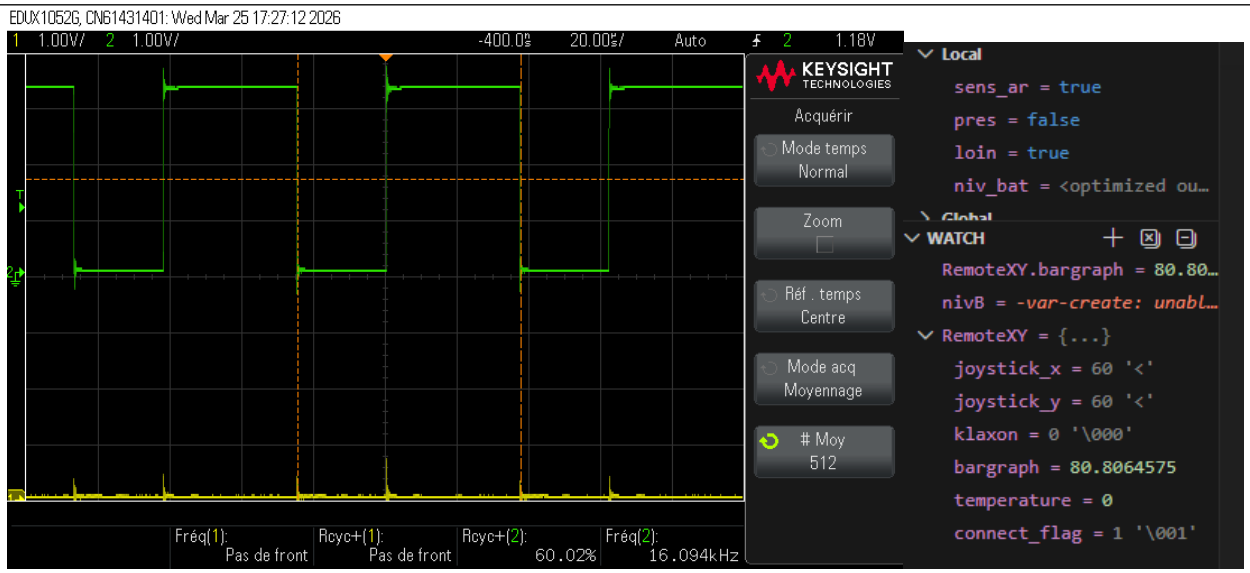
Le rapport cyclique et la fréquence des signaux PWM avant et arrière sont mesurés à l'oscilloscope.

Résultats attendus

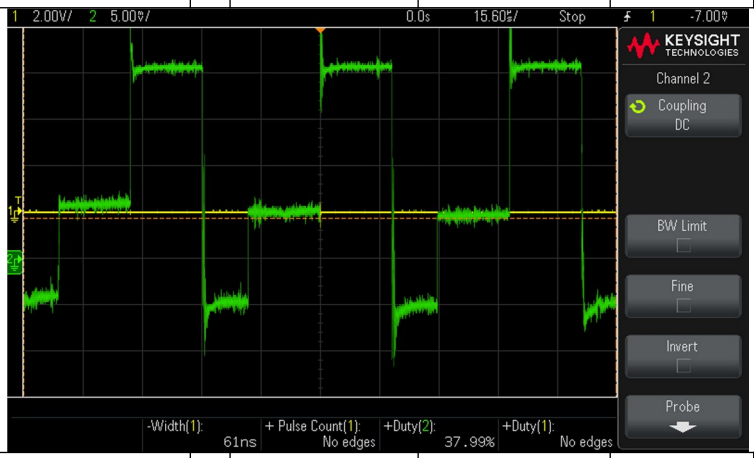
Loin	Pres	Sens_AR	RemoteXY.Y		Rapport cyclique PWM Avant	Rapport cyclique PWM arriere	Fréquence	Remarque
0	0	x	-10< RemoteXY.Y <10		0	0	0	Arrêt
0	0	x	RemoteXY.Y > 10		RemoteXY.Y	0	20khz	Marche avant
0	0	x	-10 > RemoteXY.Y		0	- RemoteXY.Y/2	20khz	Vitesse réduite en M arrière
1	0	x	-10< RemoteXY.Y <10		0	0	0	Arrêt
1	0	x	RemoteXY.Y > 10		RemoteXY.Y	0	20khz	Marche avant
1	0	x	-10 > RemoteXY.Y		0	-RemoteXY.Y	20khz	Marche arrière
0	1	1	-10< RemoteXY.Y <10		**0**	**0**	**0**	**arrêt**
0	1	1	RemoteXY.Y > 10		**?**	**?**	**?**	**?**
0	1	1	-10 > RemoteXY.Y		**?**	**?**	**?**	**?**
0	1	0	-10< RemoteXY.Y <10					
0	1	0	RemoteXY.Y > 10					
0	1	0	-10 > RemoteXY.Y		**?**	**?**	**?**	**?**

Résultats **effectifs** des tests

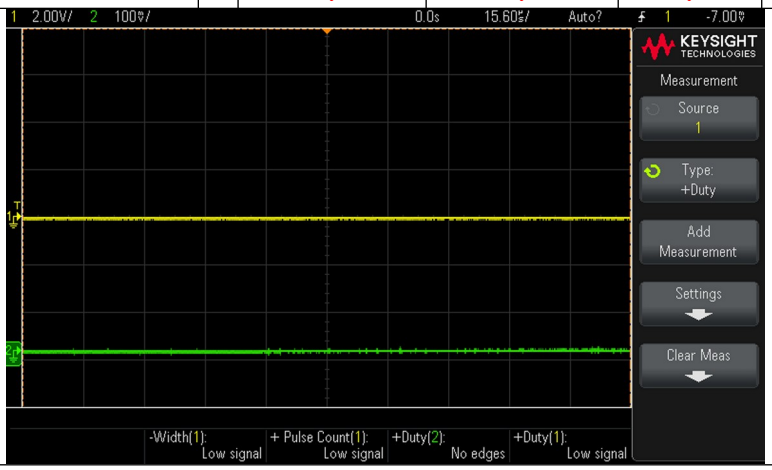
Loin	Pres	Sens_Ar	RemoteXY.Y		Rapport cyclique PWM Avant	Rapport cyclique PWM arriere	Fréquence	Remarque
0	0	x	-10< RemoteXY.Y <10		0	0	0	Arrêt
Ecran d'oscilloscope + console pour une valeur de RemoteXY								
0	0	x	RemoteXY.Y > 10		RemoteXY.Y	0	?	Marche avant
Ecran d'oscilloscope + console pour une valeur de RemoteXY								
0	0	x	-10 > RemoteXY.Y		0	- RemoteXY.Y/2	?	Vitesse réduite en M arrière
Ecran d'oscilloscope + console pour une valeur de RemoteXY								
1	0	x	-10< RemoteXY.Y <10		0	0	0	Arrêt
Ecran d'oscilloscope + console pour une valeur de RemoteXY								
1	0	x	RemoteXY.Y > 10		RemoteXY.Y	0	?	Marche avant

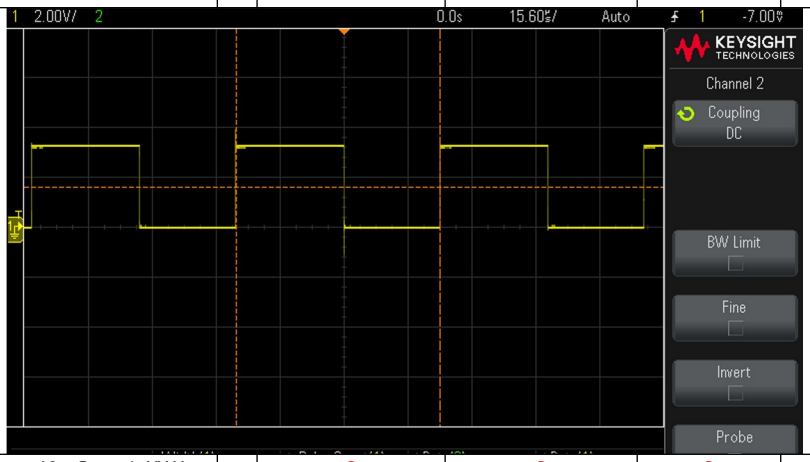


1	0	x	-10 > RemoteXY.Y		0	-RemoteXY.Y	?	Marche arrière



0	1	1	-10< RemoteXY.Y <10		?	?	?	**Zone près**



0	1	1	RemoteXY.Y > 10		?	?	?	**Anticollision**
								
0	1	1	-10 > RemoteXY.Y		?	?	?	?
Ecran d'oscilloscope + console pour une valeur de RemoteXY								
0	1	0	X		?	?	?	?
Ecran d'oscilloscope + console pour une valeur de RemoteXY								

Conclusion :

Le test est concluant. Les relations entre rapport cyclique des PWM, la consigne du joystick et les informations ……sont exactes.
La fonction est validée.

Test d'intégration

Brancher le connecteur PWM et tester la propulsion dans les différents cas du tableau précédent.
Appeler l'enseignant pour validation.

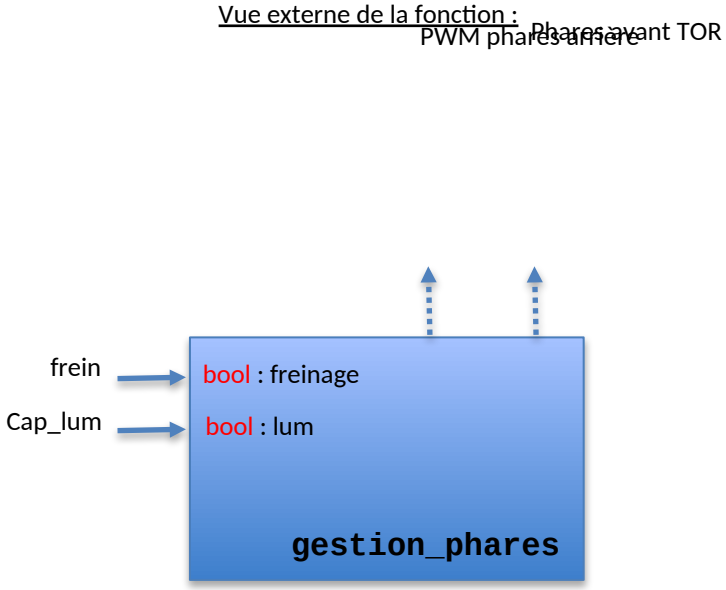
5) test unitaire & Test d'intégration de la fonction gestion_phares

Rôle de la fonction :

Dans cette fonction, les phares arrière s'allument à 30% (PWM) lorsque les phares avant sont allumés et à 100% si freinage même si les phares avant sont éteints. L'indication de freinage est fournie par la fonction propulsion.

Les phares s'allument automatiquement en cas de faible luminosité. Le seuil entre nuit et jour est de 1.5V (revoir le chapitre de capteur de luminosité de la partie électronique.

Vue externe de la fonction :



Objectif du test :

Montrer que :

- les phares avant s'allument lorsque la luminosité est insuffisante
- Les phares arrières s'allument avec un rapport cyclique de 30% si la luminosité est insuffisante et 100% si freinage quelque soit la luminosité
- La fréquence du signal PWM_PHARES_ARRIERE est de ……

Environnement du test

Ajouter ou supprimer des lignes suivant votre test

- La carte Shield est en place sur la carte microcontrôleur de la voiture.
- Le module Bluetooth est en place sur la carte Shield
- Le connecteur d'alimentation est en place sur la carte Shield.
- Le smartphone est connecté avec l'application RemoteXY.
- Le programme est chargé sur la carte microcontrôleur.
- L'environnement de débuggage **est** lancé.
- Le signal PHARES_ARRIERE est visualisé à l'écran de l'oscilloscope sur la voie1 pour mesurer la fréquence et les rapports cycliques.
- Le signal PHARES_AVANT est mesuré sur la voie2 de l'oscilloscope

Jeux de test

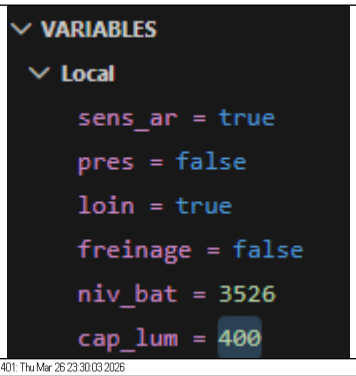
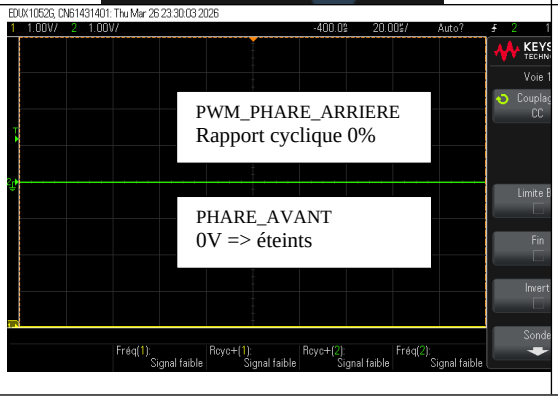
- Les variable "freinage" et "cap_lum" sont modifiée dans le debuggeur avant l'appel de la fonction

Résultats attendus

Luminosité ambiante	Luminosité suffisante		Luminosité insuffisante	
Cap_lum	< ?		>?	
Freinage (niveau logique)	"0"	"1"	"0"	"1"
PHARES_AVANT (tension)	"0" ou 0V	"0" ou 0V	"1" ou 5V	"1" ou 5V
PWM_PHARES_ARRIERE (Rapport cyclique)	0%	100%	30%	100%

La fréquence du signal PWM_PHARES_ARRIERE est de ……

Résultats des tests

Luminosité ambiante	Luminosité suffisante	
Cap_lum freinage		
PHARES_AVANT (jaune) PWM_PHARES_ARRIERE (vert)		

Luminosité ambiante	Luminosité insuffisante	
Cap_lum freinage		
PHARES_AVANT (jaune) PWM_PHARES_ARRIERE (vert)		

La fréquence du signal PWM est de ……..

Conclusion :

La fréquence attendue de …… est bien celle mesurée à l'oscilloscope. Les rapports cycliques et les tensions mesurés à l'oscilloscope sont bien ceux attendus.
La fonction est validée.

Test d'intégration

Objectif : Montrer que :

- les phares avant s'allument lorsque la luminosité est insuffisante
- Les phares arrières s'allument avec un rapport cyclique de 30% si la luminosité est insuffisante et 100% si freinage quelque soit la luminosité

  La variable cap_lum est visualisée via des `Serial.println` sur le terminal Série.

Brancher le module bluetooth sur le connecteur "con7"

Afficher dans la console la valeur de "cap_lum"

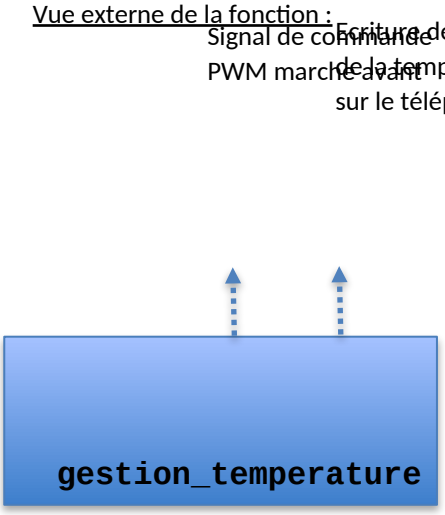
Procéder aux tests devant l'enseignant pour validation

6) Test d'intégration de la fonction `recep_temperature`

Rôle de la fonction :

- Cette fonction permet un affichage de la température extérieure sur le SMARTPHONE.
- L'adresse du capteur est personnalisée (voir les 3 cavaliers sur la carte pour l'adresse personnalisée)
- Une amélioration de la fonction est que la température ne soit rafraichie que toutes les 3 secondes.

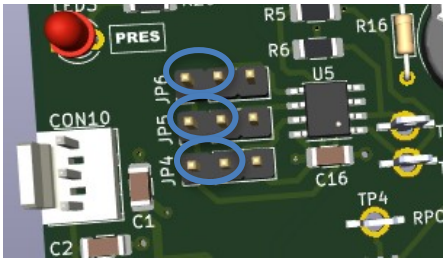
Vue externe de la fonction :



Objectif du test :
Montrer que la température ambiante est affichée sur le smartPhone

Environnement du test

- La carte Shield est en place sur la carte microcontrôleur de la voiture.
- Le module Bluetooth est en place sur la carte Shield
- Le connecteur d'alimentation est en place sur la carte Shield.
- Le smartphone est connecté avec l'application RemoteXY.
- Le programme est chargé sur la carte microcontrôleur.
- **Les cavaliers sur JP4, JP5 et JP6 respecte l'adresse personnalisée (ici l'adresse est 0x??) voir l'emplacement des cavalier ci contre.**



Résultats attendus :
La température est affichée sur le smartPhone

Résultat
Capture d'écran du smartPhone prouvant la bonne réception de la température.

Conclusion :

7) Gestion_capteur_avant

Vue externe de la fonction :



Rôle de la fonction :

Cette fonction permet de faire l'acquisition de la distance de l'obstacle à l'avant.
Pour ne pas être perturbé par les parasites électriques et obtenir une valeur suffisamment stable cette fonction fait l'acquisition de la valeur du capteur toutes les 10 ms et la range dans un tableau de 10 valeurs avec un indice (index) glissant, puis fait la moyenne sur les 10 valeurs du tableau et retourne cette valeur qui sera utilisée par ailleurs.

Objectif du test :

Montrer que la moyenne de la position est stable et qu'elle suit la loi d'évolution du capteur

Environnement du test

- La carte Shield est en place sur la carte microcontrôleur de la voiture
- Le module Bluetooth est en place sur la carte Shield
- Le connecteur d'alimentation est en place sur la carte Shield
- Le programme est chargé sur la carte microcontrôleur
- Les signaux PWM avant et arrière sont visualisés à l'écran de l'oscilloscope pour mesurer la fréquence et les rapports cycliques

Ajouter ou supprimer des lignes suivant votre test

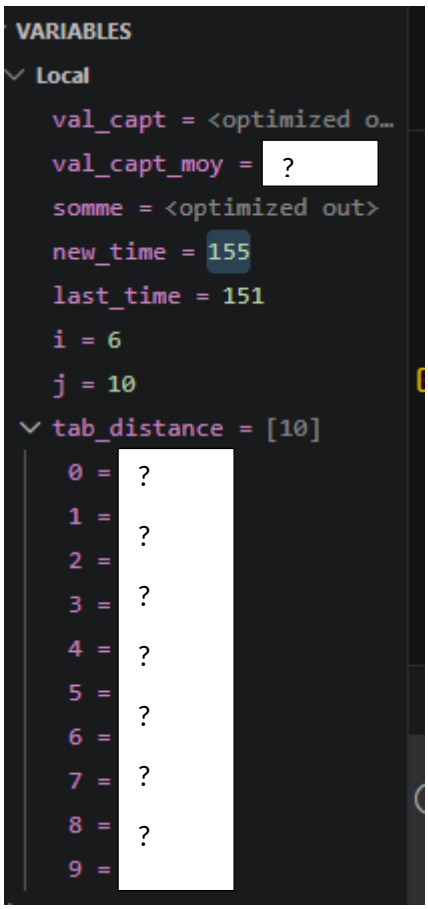
Jeux de test

- Les échantillons dans le tableau sont espionnés dans le débuggeur
- La moyenne en fonction de la distance est directement espionnée dans la console.

Résultats attendus

Distance	Moyenne attendue
20cm	
40cm	
60cm	

Test du remplissage du tableau et du calcul de la moyenne avec le debuggeur

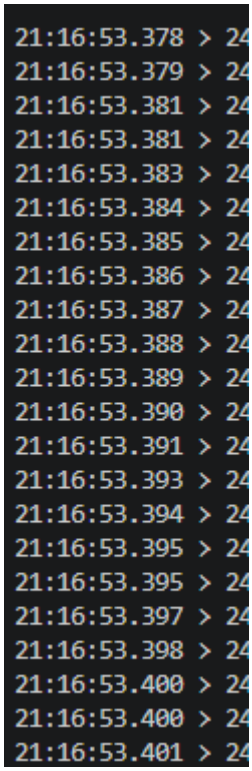


Calcul de la moyenne à la main à partir des échantillons du tableau : ?????

Comparaison avec la valeur donnée par le debuggeur :

Test de la valeur de la moyenne en fonction de la distance, dans la console

Test à X cm

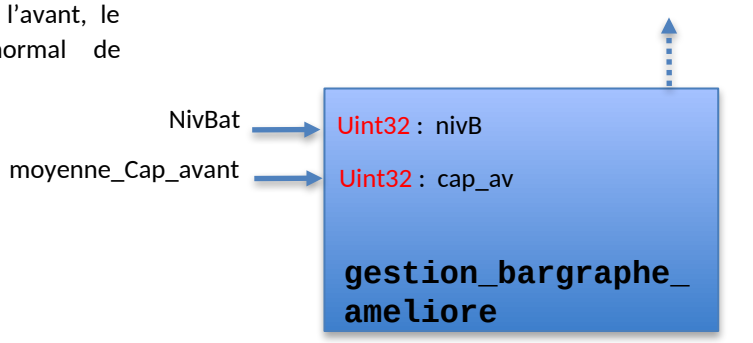


8) Gestion bargraphe améliorée

Vue externe de la fonction :

Rôle de la fonction :

Cette fonction reprend le fonctionnement du bargraphe du début. La proximité d'un obstacle avec le capteur avant est indiqué sur le bargraphe à la place du niveau de batterie lorsque la voiture est dans la zone "près".
Dès que la voiture sort de la zone "près", à l'avant, le bargraphe reprend son fonctionnement normal de l'affichage du niveau de batterie.



Objectif du test :

- Montrer la détection d'un obstacle dans la zone près à l'avant est prioritaire sur l'affichage du niveau batterie sur le bargraphe.
- Montrer que le bargraphe suit la proximité avec l'obstacle dans la zone près à l'avant.
- Montrer que le bargraphe indique le niveau de batterie si il n'y a pas d'obstacle dans la zone près à l'avant.

Environnement du test

- La carte Shield est en place sur la carte microcontrôleur de la voiture.
- Le module Bluetooth est en place sur la carte Shield
- Le connecteur d'alimentation est en place sur la carte Shield.
- Le smartphone n'est pas connecté avec l'application RemoteXY pour p
  modifier les variables.
- Le programme est chargé sur la carte microcontrôleur.
- L'environnement de débuggage **est** lancé.
- Les signaux PWM_DROITE et PWM_GAUCHE sont visualisés à l'écran de l'oscilloscope pour mesurer la fréquence et les rapports cycliques.

Ajouter ou supprimer des lignes suivant votre test

Jeux de test

Les variables niv_bat et cap_av sont modifiées dans le debuggeur lors du point d'arrêt en début de fonction :

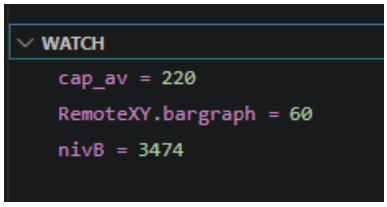
- Clic droit sur la variable à espionner puis "add to watch"
- Clic droit sur la variable dans la fenêtre "watch" pour modifier la valeur

Résultats attendus

Zone obstacle à l'avant	Loin	moyen	pres		
Cap_av	< 620	750	931	2500	3970
Niv_bat	3474 (2.8V)	3350 (2.7V)	3474	3474	3474
RemoteXY.bargraph	XX%	XX%	XX%	XX%	XX%

Résultats des tests

5 captures d'écran comme celle-ci-dessous (pour chacun des cas du tableau)



Zone obstacle à l'avant	Loin	moyen	pres		
Cap_av	< 620	750	931	2500	3970
Niv_bat	3474 (2.8V)	3350 (2.7V)	3474	3474	3474
RemoteXY.bargraph					

Conclusion :

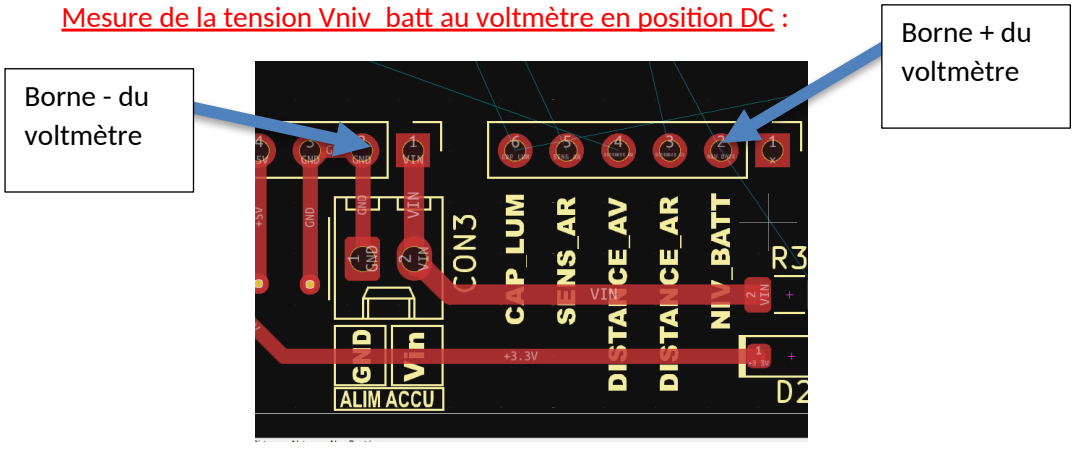
Test d'intégration

Objectif :

- Montrer la cohérence entre le niveau du bargraphe et la distance avec l'obstacle dans la zone près.
- Monter, qu'en dehors de la zone près avant, le bargraphe indique le niveau batterie (s'aider du test d'intégration de la fonction "gestion bargraph")

  La variable RemoteXY.bargraph est visualisée via des `Serial.println` sur le terminal Série.

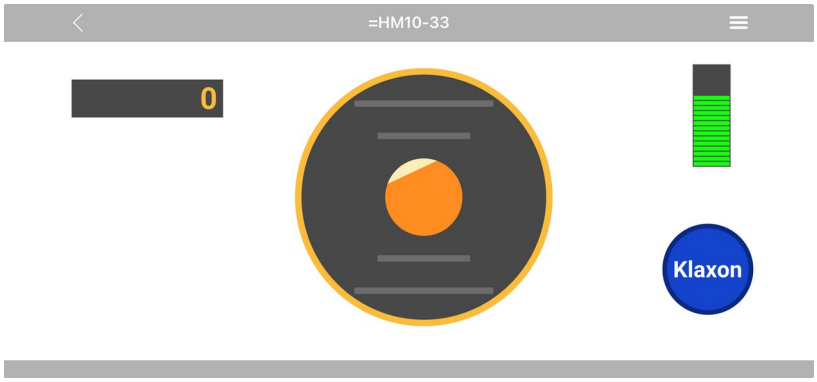
Mesure de la tension Vniv_batt au voltmètre en position DC :



Résultat de la mesure : Vniv_batt = XXV

conversion de cette tension en pourcentage du bargraph : (XX*4095/3.3)*100/620 – 500 = XX,X%

Copie d'écran du smartphone prouvant le bon fonctionnement du bargraph



14 barres sont allumées sur 20 ce qui correspond à

XX/YY*ZZ = JJ%

Valeur en accord avec la mesure précédente car l'affichage affiche JJ% pour une valeur réelle entre TT% et MM%

9) Gestion sens avant

Vue externe de la fonction :

Rôle de la fonction :

Cette fonction utilise la valeur moyenne retournée par la fonction `gest_capt_avant` pour retourner, elle-même, une valeur indiquant une approche d'obstacle à l'avant.
Toutes les 100 ms une comparaison est effectuée. Si la valeur moyenne de la position a augmenté de plus de 5 alors le signal d'approche retourné par la fonction vaut `true` sinon `false`.

Test d'intégration

Objectif du test :

- Montrer la fonction renvoie "1" lorsque le véhicule est en approche de l'obstacle
- Montrer la fonction renvoie 0 lorsque le véhicule est à l'arrêt ou s'éloigne de l'obstacle

Environnement du test

- La carte Shield est en place sur la carte microcontrôleur de la voiture.
- Le module Bluetooth est en place sur la carte Shield
- Le connecteur d'alimentation est en place sur la carte Shield.
- Le programme est chargé sur la carte microcontrôleur.

Ajouter ou supprimer des lignes suivant votre test

Jeux de test

La variable "sens_av" est espionnée dans la console

Résultats attendus

	Capture d'écran de la console
Véhicule en approche d'obstacle	
Véhicule à l'arrêt	
Véhicule qui s'éloigne de l'obstacle	

Tests devant l'enseignant pour validation.

10) Fonction gestion buzzer ameliore

Rôle de la fonction :

Vue externe de la fonction :

.

Objectif des tests :

Montrer que les émissions sonores du buzzer répondent au cahier des charges (vérifier les timings et les priorités)

Environnement du test

- La carte Shield est en place sur la carte microcontrôleur de la voiture
- Le module Bluetooth est en place sur la carte Shield
- Le connecteur d'alimentation est en place sur la carte Shield
- Le smartphone n'est pas connecté avec l'application RemoteXY
- Le programme est chargé sur la carte microcontrôleur
- L'environnement de débuggage n'est pas lancé car influence sur la fonction millis()
- Le connecteur du buzzer est en place sur la carte Shield
- Oscilloscope pour la mesure des temps du signal …….

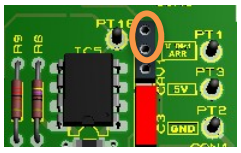
Ajouter ou supprimer des lignes suivant votre test

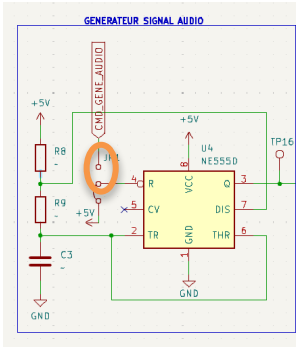
Jeu de tests

Nous testons uniquement l'influence du capteur avant. Les valeurs des variables `pres`, `loin`,`cap_av` et `RemoteXY.klaxon` sont imposées dans le loop (Après les acquisitions, pour écraser les valeurs de l'environnement). Ne pas oublier de Supprimer ces lignes de code après le test.
Pour des raisons de temps nous ne testerons que le fonctionnement du capteur avant avec pres=0 et loin =1.

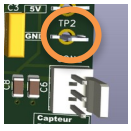
Utilisation de la maquette directement dans l'environnement avec visualisation du signal **Cmd_Gene_Audio** sur l'oscilloscope pour mesure les temps et allure du signal

Borne+ de l'oscilloscope sur le cavalier de JP1 (ou cav1 suivant la carte), placé en position haute





Borne – de l'oscilloscope sur TP2 (GND de la carte)



Résultats **attendus**

Cap_av	RemoteXY.klaxon		Cmd_Gene_Audio	Remarques
<620	0		0V	Sans obstacle à moins de YY cm du capteur et sans klaxon => Pas de son
≥620 et ≤930	0		Signal carré de demi-période YYY ms	Obstacle entre XX et YY cm du capteur et sans klaxon => X bips par seconde
>930	0		Signal carré de demi-période YYY ms	Obstacle entre XX et YY cm du capteur et sans klaxon => J bips par seconde
x	1		X,X V	Son continu pour le klaxon

Résultats **effectifs** des tests

Cap_av	RemoteXY.klaxon		Cmd_Gene_Audio	Remarques
500	0			Obstacle à plus de ????????? et sans klaxon => ?????????

Capture d'oscilloscope

Commentaire

Jeu de test

```
// Jeux de test buzzer
  pres = 0;
  loin = 1;
  cap_av = 500;
  RemoteXY.klaxon = 0;
```

800	0		????????	Obstacle entre ???? et ???? cm du capteur et sans klaxon => ???? bips par seconde

Mesures + commentaires

Calcul de l'erreur

Jeu de test

1000	0		???????????	Obstacle à moins ??????? du capteur et sans klaxon => ??? bips par seconde

Mesures + commentaires + calcul de l'erreur

Jeu de test



500 800 1000	1			L'appui sur le klaxon est prioritaire sur les bips

Commentaire

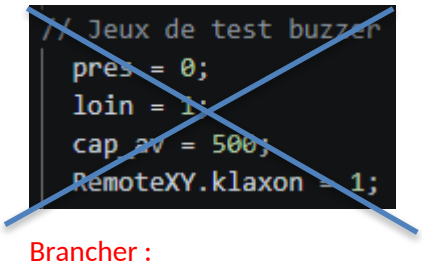
Jeu de test
3 jeux de test ici

Conclusion :

Test d'intégration

Objectif : montrer que le comportement du buzzer répond au cahier des charges

Décommenter ce qui avait été commenté pour les jeux de test unitaire et suppression des lignes suivantes :



Brancher :

- Le capteur arrière sur le connecteur "??"
- Le capteur avant sur le connecteur "??"
- Le buzzer sur le connecteur "??"
- Le module bluetooth sur le connecteur "??"

Procéder aux tests de zone et de priorité du klaxon avec le son du buzzer, devant l'enseignant.

Klaxon	Cap_av	pres	loin		BUZZER
1	x	x	x		bip continu
0	Entre 40cm et 60cm à l'avant	0	1		? bips par seconde
0	Plus de 40cm à l'avant	0	0		? bips par seconde
0	Plus de 60cm à l'avant (<620)	0	1		Pas de son
0	>930	x	0		? bips par seconde
0	x	1	0		? bips par seconde

11) Fonction propulsion améliorée

Rôle de la fonction :

Vue externe de la fonction :

Objectif du test :

Montrer que la voiture réagit bien en fonction des consignes du joystick et des différentes zones avant et arrière. Seul le test d'intégration sera effectué pour des raisons de temps.

Environnement du test

- La carte Shield est en place sur la carte microcontrôleur de la voiture
- Le module Bluetooth est en place sur la carte Shield
- Le connecteur d'alimentation est en place sur la carte Shield
- Le programme est chargé sur la carte microcontrôleur

Ajouter ou supprimer des lignes suivant votre test

Jeux de test

La consigne RemoteXY sur l'axe Y est modifiée par action sur le joystick

La variable de connection bluetooth "connect_flag"

Les variables "pres", "loin", "sens_ar", "sens_av" et "cap_av" sont modifiées directement dans l'environnement extérieur de la voiture

Résultats attendus

Loin	Pres	Sens_AR	Sens_av	Cap_av	Connect_flag	RemoteXY.Y	Mode de marche	Rapport cyclique PWM Avant	Rapport cyclique PWM arriere	Fréquence	Remarque
0	0	x	X	≤ 930	1	-10< RemoteXY.Y <10	Roue libre	0	0	0	Arrêt
0	0	x	0	> 930	1	-10< RemoteXY.Y <10		0	0	0	Arrêt
x	0	x	1	> 930	1	-10< RemoteXY.Y <10		0	100	?	Anticollision
x	1	0	0	x	1	-10< RemoteXY.Y <10		100	0	?	Anticollision
x	1	1	x	x	1	-10< RemoteXY.Y <10		100	0	?	Arrêt
x	x	x	X	≥ 620 ET ≤ 930	1	RemoteXY.Y > 10	Avant	RemoteXY.Y/2	0	?	Vitesse réduite en Marche avant
x	x	x	X	< 620	1	RemoteXY.Y > 10		RemoteXY.Y	0	?	Marche avant
x	x	x	1	> 930	1	RemoteXY.Y > 10		RemoteXY.Y	0	?	Anticollision
x	x	x	0	> 930	1	RemoteXY.Y > 10		RemoteXY.Y	0	?	Arrêt
0	0	x	X	x	1	-10 > RemoteXY.Y	Arrière	0	- RemoteXY.Y/2	?	Vitesse réduite en M arrière
1	0	x	x	x	1	-10 > RemoteXY.Y		0	-RemoteXY.Y	?	Marche arrière
0	1	1	x	x	1	-10 > RemoteXY.Y		0	0	0	Arrêt
0	1	0	x	x	1	-10 > RemoteXY.Y		100	0	?	Anticollision
x	x	x	x	x	0	x		0	0	0	Perte connexion

Remarque :

Cap_av < 620 => obstacle loin à l'avant
Cap_av > 930 => obstacle près à l'avant

Résultats **effectifs** des tests

Tester la propulsion dans les différents cas du tableau précédent.
Appeler l'enseignant pour validation.

Conclusion :

12) Fonction gestion buzzer pwm

Rôle de la fonction :

Vue externe de la fonction :

Objectif du test :

Montrer que les émissions sonores du buzzer répondent au cahier des charges (vérifier les timings, les priorités et les fréquences du signal)

Environnement du test

- La carte Shield est en place sur la carte microcontrôleur de la voiture
- Le module Bluetooth est en place sur la carte Shield
- Le connecteur d'alimentation est en place sur la carte Shield
- Le programme est chargé sur la carte microcontrôleur

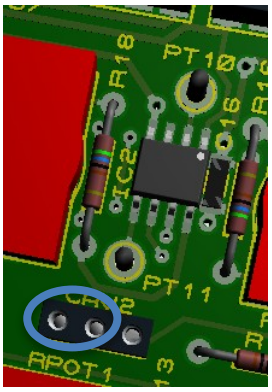
Ajouter ou supprimer des lignes suivant votre test

Jeux de test

Les variables "pres", "loin", "cap_av" et "RemoteXY.klaxon" sont modifiées directement dans l'environnement extérieur de la voiture.

Le signal TONE est visualisé à l'oscilloscope. La mesure s'effectue sur CAV2

Placer le cavalier de CAV2 à gauche afin que le signal du buzzer soit le signal TONE et Non le signal de l'oscillateur



Résultats attendus

Loin	Pres	Cap_av	Klaxon	Zone de proximité	Caractéristique du Signal TONE
0	0	≤ 930	0	Moyen à l'arrière	Signal tone de fréquence 3500Hz avec une enveloppe de demi période 250ms
x	0	> 930	0	Près à l'avant	Signal tone de fréquence 3500Hz avec une enveloppe de demi période 125ms
x	1	x	0	Près à l'arrière	Signal tone de fréquence 3500Hz avec une enveloppe de demi période 125ms
1	0	≥ 620 ET ≤ 930	0	Moyen à l'avant	Signal tone de fréquence 3500Hz avec une enveloppe de demi période 250ms
1	0	< 620	0	loin	0V
x	x	x	1	Klaxon	3.3V

Résultats **effectifs** des tests

Loin	Pres	Cap_av	Klaxon	Zone de proximité	Capture d'écran de l'oscilloscope avec toutes les mesures
0	0	≤ 930	0	Moyen à l'arrière	
x	0	> 930	0	Près à l'avant	
x	1	x	0	Près à l'arrière	
1	0	≥ 620 ET ≤ 930	0	Moyen à l'avant	
1	0	< 620	0	loin	
x	x	x	1	Klaxon	

Appeler l'enseignant pour validation.

Test d'intégration pour la cucaracha

Objectif : montrer que le comportement du buzzer répond au cahier des charges

Appeler l'enseignant

F. Test d'Intégration des fonctions

Fait avec les tests unitaires plus haut

Annexes

Schémas Electriques

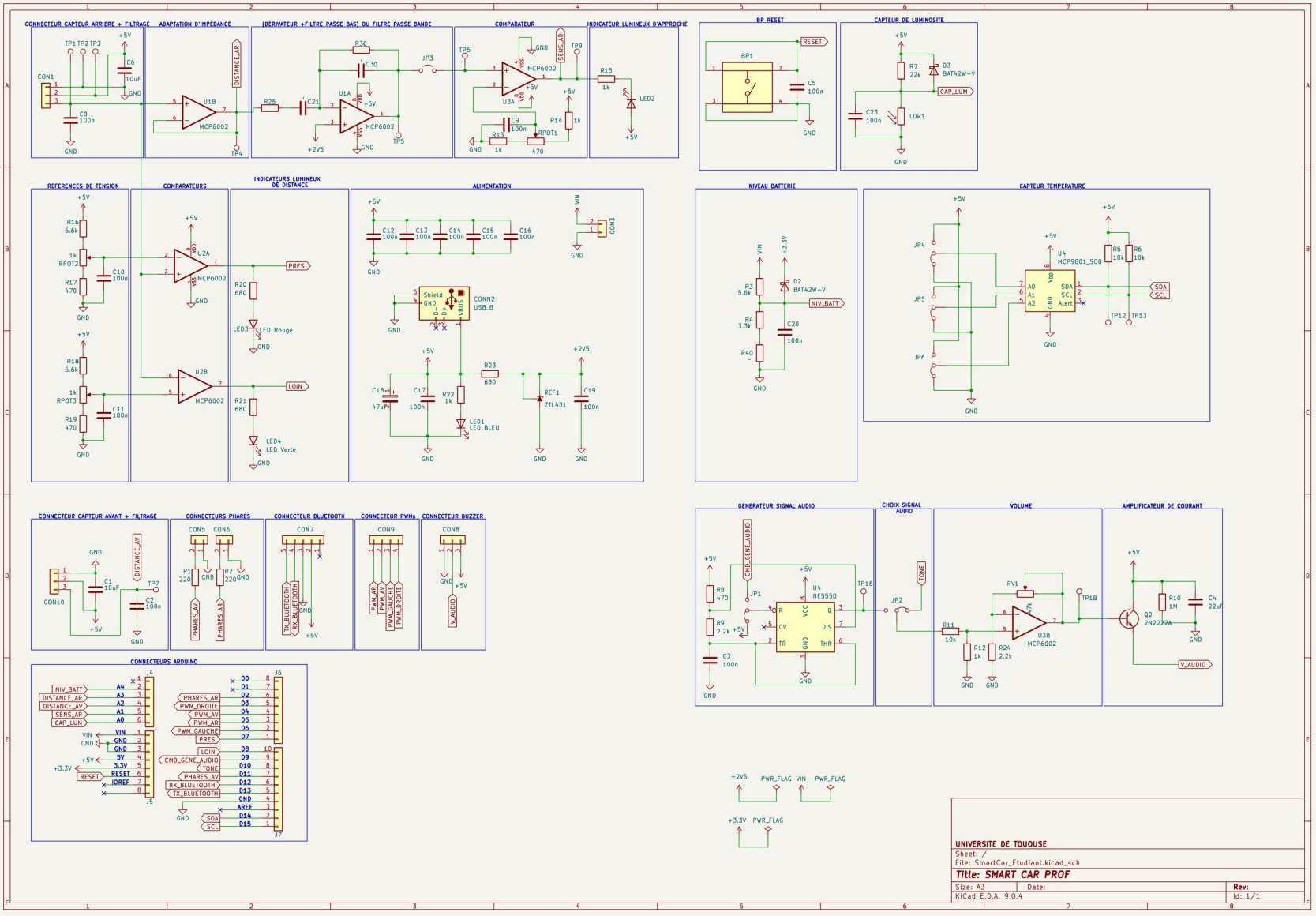


Schéma d'Implantation de la carte Shield

